Evolutionary-Based Neural Architecture Search Optimisation for Sustainable Deep Learning Model

Kian Lee

4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2024

Abstract

With modern technological advancements, the sustainability issue associated with utilising deep neural networks persists as a pressing concern. The optimisation of the neural architecture becomes imperative for ensuring the long-term viability and efficacy of neural networks. Extensive research indicates a notable lack of attention to sustainability issues within the field. It is observed that optimising genetic algorithms within Evolutionary Algorithm-based Neural Architecture Search possesses the potential to address the aforementioned sustainability concerns. An established NAS framework GeneticNAS proposed by Habi and Rafalovich [2019] is investigated and novel methodologies are proposed to fine-tune its genetic algorithms. Through experimentation, enhancements in final validation accuracy and search process efficiency are demonstrated. Specifically, Adaptive Flip v.2 emerges as a promising approach for efficiency, while Blend Crossover significantly boosts accuracy with some minor compromises in model complexity. The integration of these two methodologies provides intriguing insights. While there is a significant performance improvement observed for the CIFAR10 classification, such gains are not mirrored in CIFAR100. This suggests the necessity for further optimisation, particularly in adjusting mutation rates for deeper networks. The findings underscore the potential of fine-tuning genetic algorithms within NAS systems to address sustainability concerns and enhance model performance, offering avenues for future research and development in this domain.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Kian Lee*)

Acknowledgements

I wish to express my heartfelt gratitude to my project supervisor, Dr Michael Herrmann, for his steadfast support and mentorship throughout this academic endeavour. His invaluable guidance and insightful feedback have profoundly influenced the direction of my research and contributed significantly to the completion of this project.

I would also like to extend my appreciation to David Symons for his constructive feedback and valuable advice. Without his input, this thesis would not have reached its current level of refinement.

Table of Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Question . . . 2
1.4 Hypothesis . . . 3
1.5 Outcomes and Contributions . . . 3
1.6 Thesis Structure . . . 3

2 Background **4**
2.1 Neural Network . . . 4
2.2 Sustainability of Deep NN . . . 4
2.3 Convolutional Network . . . 5
2.3.1 Convolutional layer . . . 5
2.3.2 Pooling Layer . . . 6
2.3.3 Fully-Connected Layer . . . 7
2.3.4 CNN Complexity . . . 7
2.4 Neural Architecture Search . . . 7
2.4.1 Reinforcement Learning . . . 8
2.4.2 Evolutionary Algorithm . . . 8
2.5 One-shot NAS . . . 9
2.6 Deep Learning Optimisation Strategies . . . 9
2.6.1 Data Pre-processing and Augmentation . . . 10
2.6.2 Multi-step Learning Rate . . . 10
2.6.3 Dropout and drop-path . . . 10
2.7 Potential Optimisation . . . 10

3 Methodologies **11**
3.1 GeneticNAS . . . 11
3.1.1 Search Space . . . 11
3.1.2 Network Architecture . . . 14
3.1.3 Neural Architecture Search . . . 14
3.2 Experiments . . . 18
3.2.1 Gaussian Flip . . . 19
3.2.2 Uniform Random . . . 20
3.2.3 Adaptive Flip . . . 20

3.2.4 Blend crossover 21

4 Results **23**
4.1 CIFAR10 25
4.1.1 Overview of Models 25
4.1.2 Validation 26
4.1.3 Model Complexity 29
4.1.4 Efficiency 30
4.2 CIFAR100 31
4.2.1 Overview of Models 31
4.2.2 Validation 32
4.2.3 Model Complexity 35
4.2.4 Efficiency 36

5 Discussions **37**
5.1 Motivation and Rationale 37
5.2 Discoveries 38

6 Conclusions **39**
6.1 Limitation 39
6.2 Future work 40
6.3 Conclusion 40

Bibliography **41**

A Appendix **45**

Chapter 1

Introduction

1.1 Motivation

Machine learning has become increasingly ubiquitous in modern life, seamlessly integrating into diverse domains such as autonomous vehicles [Vishnukumar et al., 2017] and image recognition [Wu and Chen, 2015]. The aforementioned domains heavily rely on deep learning which is a subset of the machine learning algorithm neural network(NN). Deep learning enables neural network computational models comprising multiple hierarchical processing layers to learn the representation of data in multiple levels of abstractions [LeCun et al., 2015].

In the realm of machine learning, the efficient utilisation of neural networks has been a preeminent concern, primarily due to the ever-increasing demands for computational resources, power consumption, and memory usage. As we move further into the digital age, the requirement for sustainable and environmentally friendly machine learning practices becomes a pressing concern [Thompson et al., 2020].

Power consumption remains a critical concern in the field of neural networks, particularly during the training and inference phases of deeper networks. The utilisation of high-performance hardware, such as Graphics Processing Units (GPUs) and Tensor Processing Units (TPUs), has become commonplace in machine learning. However, the excessive power consumption associated with these hardware components raises environmental and cost-related issues. The largest language model GPT-3 launched by OpenAI takes 4.6 million USD and 355 years in computing time if trained on ordinary GPU [Martineau, 2020].

Most deep NNs are designed as complex and deep as possible with the goal of achieving the maximum possible accuracy and completely disregarding the issue of power consumption [García-Martín et al., 2019].

1.2 Problem Statement

In order to address the outlined sustainability issue, it is imperative to optimise the utilisation of NNs. The most salient and logical solution would be optimising the extremely complex architecture of these networks. However, handcrafting neural network architecture to strike the optimum trade-off for accuracy and efficiency is often impractical, especially for very deep and complicated networks which require a substantial amount of prior knowledge of the network. Thus, neural architecture search (NAS) was invented where the search for optimal neural network architecture is automatically done by algorithms [Ren et al., 2021].

In recent times, GeneticNAS [Habi and Rafalovich, 2019] was proposed. GeneticNAS utilises genetic algorithms coupled with conventional Stochastic Gradient Descent (SGD) and weight-sharing in the autonomic search for neural architecture. In this work, The genetic algorithm used in the cited GeneticNAS is optimised to attain a better accuracy-efficiency trade-off. The behaviour of modifying the genetic algorithm strategies will be monitored and evaluated in terms of best model validation accuracy, complexity and elapsed time to train(efficiency).

Habi and Rafalovich's GeneticNAS is employed in this project because of the following rationale.

- GeneticNAS integrates the favourable attributes of state-of-the-art NAS, known as efficient neural architecture search (ENAS) and further enhances the capacity of architecture search by incorporating evolutionary algorithms.
- GeneticNAS utilises common machine learning technique dropout to prevent over-fitting phenomenon and multi-step learning rate to facilitate efficient convergence of solution during training.
- GeneticNAS utilises techniques such as randomisation selector to remove computational overhead while still allowing effects to take place. This contributes to addressing the sustainability concern mentioned.

Every technique mentioned will be discussed further in chapter 2 - background.

1.3 Research Question

However, GeneticNAS solely focused on the optimisation of parameters within their proposed genetic algorithm methodology. The comparative effectiveness or efficiency of alternative evolutionary algorithms lacks investigation.

Thus, the research question of this project is:

> Investigate alternative evolutionary algorithms and scheduling strategies to maximise the accuracy of GeneticNAS, without compromising too much efficiency.

The objective is to strike a balance between accuracy maximisation and efficiency trade-off. The exact methodology will be discussed in detail in chapter 3 - methodology.

1.4 Hypothesis

The hypothesis of this project is:

> By fine-tuning genetic algorithms within an evolutionary algorithm-based NAS system, the trade-off between accuracy and efficiency will be improved.

1.5 Outcomes and Contributions

Optimising genetic algorithms within an Evolutionary Algorithm-based NAS system demonstrated potential as a viable strategy for enhancing both the efficiency and accuracy of the NAS system. It holds the potential to alleviate the aforementioned sustainability concerns.

This project makes the following contributions:

- Provided a direction for further investigation in the emerging field of neural architecture search.
- Further optimised an established NAS system GeneticNAS and evaluated the trade-off between accuracy and efficiency.
- Provided a pipeline and direction to optimise EA-based NAS for future work.
- Discovered a potential NAS optimisation tailored to combat sustainability issues.

1.6 Thesis Structure

- **Chapter 1** serves as the project's introductory section, highlighting the imperative of NAS and outlining the central research question.
- **Chapter 2** serves as the foundational background section, offering essential insights into the requisite knowledge for comprehending the experimental procedures and methodologies employed. Furthermore, it encompasses relevant literature and contextualises the research within the broader academic discourse.
- **Chapter 3** serves as the experiment methodology section, outlining the architecture search procedure employed by GeneticNAS and clarifying the novel modifications made to the original genetic algorithm strategy.
- **Chapter 4** serves as the result section, presenting and discussing the results obtained in the experiments. Additionally, it establishes the evaluation metrics utilised for assessing the efficacy of the proposed approach.
- **Chapter 5** serves as the discussion section, discussing the rationales of the proposed project while critically analysing the discoveries.
- **Chapter 6** serves as the conclusion section, concluding the findings of the proposed project while addressing limitations and providing direction of future work.

Chapter 2

Background

In this chapter, the essential information and concepts requisite for understanding the experimental protocols and methodologies implemented in the project are provided. The research is also contextualised within the broader academic discourse by shedding light on literature that is relevant to the project.

2.1 Neural Network

Neural network(NN) is inspired by the human biological nervous system [Krogh, 2008]. The neural network architecture is composed of input layers, hidden layers and output layers. Each layer can have a different number of nodes that are often referred to as neurons. Each neuron has a bias(weight) associated with subsequent neurons. NN with three or more hidden layers are considered deep neural networks.

2.2 Sustainability of Deep NN

Deep neural networks with a vast number of parameters demand substantial resources for storage and computation. Image classification for extremely complex and large datasets often requires deep neural networks with intricate architecture and extensive depth which is voraciously demanding on computational power and very unsustainable.

In 2020, The CTO of Advanced Micro Devices(AMD) Mark Papermaster displayed a slide at Design Automation Conference showcasing that machine learning system's power consumption is predicted to surpass the world's power production in the future, mainly because of neural networks [Bailey, 2022] (See Figure 2.1).

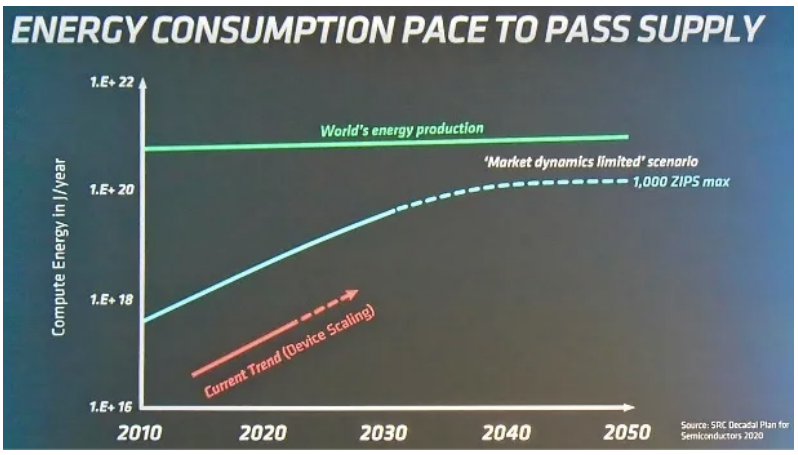

Figure 2.1: The power consumption trend of machine learning systems [Bailey, 2022]

2.3 Convolutional Network

The two mainstream deep neural networks utilised in the field of computer vision are the recurrent neural network (RNN) and the convolutional neural network (CNN). CNN is a type of feed-forward NN that is capable of extracting features of data through convolution structures. A CNN typically comprises three primary layers: a convolutional layer, a pooling layer, and a fully connected layer [Li et al., 2022]. State-of-art CNN architectures such as ResNet (Residual Networks) [He et al., 2015], EfficientNet [Tan and Le, 2019] and Inception (GoogLeNet) [Szegedy et al., 2014] made a significant contribution to the field of computer vision.

The application of CNN can be traced back to the late 1980s when the advancement of such an approach remained dormant due to a lack of computational power and resources [Rawat and Wang, 2017]. CNNs have demonstrated remarkable success across various domains within computer vision. A notable recent advancement in image classification is MobileNets, recognised for its efficiency in mobile and embedded vision applications [Howard et al., 2017]. Moreover, CNNs have excelled in object detection tasks, evidenced by models like fast region-based convolutional network (Fast CNN) [Girshick, 2015] and single shot multi-block detector [Liu et al., 2016]. Additionally, they have made significant strides in semantic segmentation such as the *fully convolutional network* [Long et al., 2015] and mask R-CNN [He et al., 2017].

2.3.1 Convolutional layer

The convolutional layer (CONV) utilises filters to conduct convolution operations by scanning the input data I according to its dimensions. It is characterised by hyperparameters such as the filter size F and the stride S. The output O known as the feature map or activation map, is generated through this process [Amidi and Amidi, 2019]. The CONV layer converts images into numerical values for NN to interpret and extract relevant patterns. CNN offer significant utility and practical value in various applications, including the diagnosis of autism spectrum disorder using convolutional autoencoders and structural MRI images [Mostafa and Wu, 2021].

The quantity of filters influences the depth of the output. Employing x separate filters would generate x distinct feature maps, thereby resulting in a depth of x. The stride refers to the distance or the number of pixels that the kernel traverses across the input matrix. Although stride values of two or higher are uncommon, employing a larger stride leads to a reduction in the output size. Following each convolution operation, a CNN typically applies a Rectified Linear Unit (ReLU) transformation to the feature map, thereby introducing non-linearity to the model.

Depthwise Separable Convolution is a convolutional operation that decomposes the standard convolution process into two distinct steps proposed by Chollet [Chollet, 2017]. The convolution operation of depthwise separable convolution can be simplified into two steps:

- Depthwise convolution applies a single convolutional filter independently to each input channel.
- Pointwise convolution combines the output channels from the depthwise convolution through a linear combination.

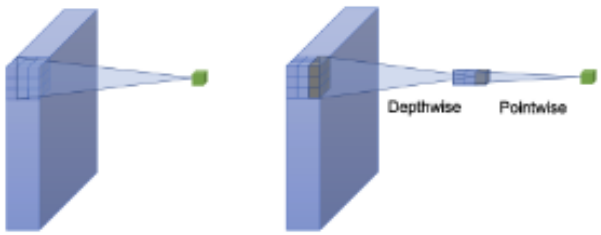

Figure 2.2: Standard convolution operation (Left) vs Depthwise Separable convolution (Right). Taken from [PapersWithCode, Accessed 2024]

This approach effectively reduces computational complexity and model parameters compared to standard convolution while improving performance in various tasks.

2.3.2 Pooling Layer

Pooling layers, often referred to as downsampling, perform dimensionality reduction on input by decreasing the number of parameters within the input. The two principal types of pooling include max-pooling and average-pooling:

- Max pooling involves selecting the pixel with the highest value as the filter traverses the input, transmitting it to the output array [IBM, 2023]. This is a more common pooling method than average pooling.
- Average pooling involves computing the mean value within the receptive field as the filter moves across the input, subsequently transmitting it to the output array[IBM, 2023].

A pooling layer typically has no trainable parameter.

2.3.3 Fully-Connected Layer

The fully-connected layer is self-explanatory with its name. In the fully-connected layer, each node in the output layer establishes direct connections with nodes in the preceding layer. This layer undertakes the classification task by utilising the features extracted from preceding layers along with their respective filters.

2.3.4 CNN Complexity

The trainable parameter of a CNN can be calculated by:

No. trainable parameters of CONV layer + No. trainable parameters of fully connected layer

The trainable parameter of a convolutional layer can be calculated by:

$$\text{\# Parameters} = (\text{filter width(n)} \times \text{filter height(m)} \times \text{input channels}(l) + 1) \times \text{number of filters(k)}$$

The trainable parameter of a fully connected layer can be calculated by

$$\text{\# Parameters} = n \times m + m$$

The evaluation of a Convolutional Neural Network's complexity often revolves around its trainable parameters. A CNN's complexity increases proportionally with its trainable parameter count.

2.4 Neural Architecture Search

Neural Architecture Search(NAS) aims to explore task-specific NN architectures that exhibit high performance on a designated dataset, thereby liberating engineers from the time-consuming labour of designing network architectures manually. The process can be conceptualised as a bi-level optimisation problem as shown in 2.1 [Liu et al., 2022]:

$$A^* = \underset{W}{\operatorname{argmin}} L_{val}(N(A, W_A^*), D_{val}) \tag{2.1}$$

$$W_A^* = \underset{A \in S}{\operatorname{argmin}} L_{tra}(N(A, W_A), D_{tra}) \tag{2.2}$$

Where D_{tra}, D_{val} corresponds to designated training and validation dataset. $N(A, W_A)$ denotes the architectural candidate within the search space. W_A denotes the parameters associated to the network A.

*Equations and description from Liu et al. [2022].

The objective of the NAS is to search for an architecture $A \in S$ that is capable of attaining promising performance on the validation set D_{val} through the minimisation of the loss function L_{val}, Such A is denoted as A^* and the search process can be represented by equation 2.1. The parameters W_A can be acquired by training model A on the training set D_{train}, accomplished through the minimisation of the loss function L_{train} according to equation 2.2.

The employment of NAS has demonstrated considerable success in the architectural design of language and image recognition models in recent years[Zoph and Le, 2017]. The conventional NAS that is established in the image classification field can be categorised into reinforcement learning(RL) and evolutionary algorithm(EA).

2.4.1 Reinforcement Learning

In RL-based NAS, an RNN controller undergoes training to generate potential candidate network architecture, often referred to as a *child model* and subsequently be trained to converge. Upon completion of training such child model, the performance of the model is evaluated against the dataset of interest. The controller is then updated by the evaluation outcome and the sampling for the next iteration is refined. RL-Based NAS demonstrated great success in the field of computer vision [Zoph et al., 2018]. Numerous researchers have built upon this groundwork, such as introducing MetaQNN [Baker et al., 2017] and BlockQNN [Zhong et al., 2018], both of which leverage Q-learning with an epsilon-greedy exploration strategy to develop efficient CNN architectures.

2.4.2 Evolutionary Algorithm

On the other hand, EA is a heuristic search method inspired by Darwin's evolution concept of natural selection and genetics [Galvan et al., 2003]. In the context of NAS, EA typically consists of four key building blocks: selection, cross-over, mutation and replacement. The building blocks are used to encode a neural architecture search space much like a chromosome. Each element within a chromosome is designated as a genotype(gene) while a distinct solution is denoted as an individual. Another fundamental concept of GAs is the notion of a *population* which serves as the basis for generating new candidate solutions. During each iteration, a new generation is formed through a sequence of selection, crossover, and mutation processes. This new generation is integrated into the population via a replacement step in each iteration, where better performing individuals will remain and the lesser effective ones will be removed, much like survival of the fitness in evolution [Habi and Rafalovich, 2019]. Upon completion of training, the individual with the best accuracy will be selected as the final architecture. The application of EA-based NAS have seen success in the image recognition field [Xie and Yuille, 2017], consolidated by [Miikkulainen et al., 2017] and [Real et al., 2017] demonstrating the advantage of NAS over traditional handcrafted NNs.

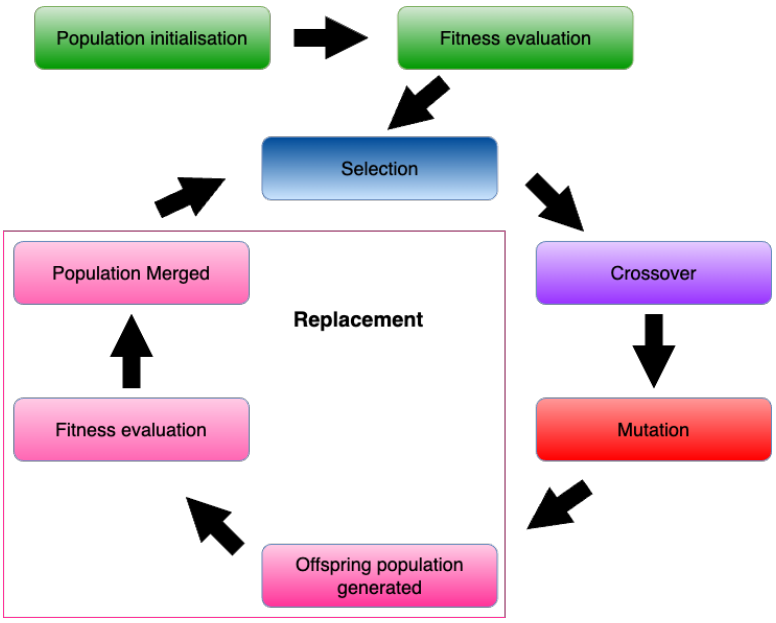

Figure 2.3: EA-based NAS framework

2.5 One-shot NAS

While both RL-based and EA-based approaches have demonstrated the ability to acquire network structures surpassing manually crafted architectures, they still require substantial computational resources [Liu et al., 2018]. For the purpose of facilitating such an issue, the idea of parameter sharing is invented, where the model uses weights from previous iterations instead of training from scratch. The common practice of parameter sharing utilised in NAS includes efficient neural architecture search(ENAS) [Cai et al., 2017] and differentiable architecture search (DARTS) [Liu et al., 2019]. ENAS is based on RL with shared weights while DARTS on the other hand formulate the search as an optimisation problem that can be solved using gradient-based optimisation algorithms. ENAS [Pham et al., 2018] pioneered a NAS training methodology featuring shared parameters[Xie et al., 2023]. This approach significantly mitigated resource requirements, reducing the computational burden from thousands of GPU days to a single one. Rather than initiating a new training for each sampled model, subsequent models inherit parameters from previously-trained ones [Yu et al., 2019]. NAS that utilises shared parameters is often referred to as one-shot NAS.

2.6 Deep Learning Optimisation Strategies

GeneticNAS by Habi and Rafalovich [2019] employs a variety of optimisation techniques that are commonly used in deep NN to address the challenge of image classification. These methods are integrated into the training phase of the NAS which enhances model generalisation and optimised the model performance. The greater the optimisation of the search strategy, the more efficient the NAS model becomes, thereby mitigating stress on the aforementioned sustainability issue.

2.6.1 Data Pre-processing and Augmentation

The experiment utilised CIFAR10 and CIFAR100 datasets, each comprising 50,000 training images and 10,000 validation images from Alex [2009]. The preprocessing and augmentation procedures employed in both datasets adhere to the methodologies established in a prior work He et al. [2015].

The data is pre-processed by normalisation on all images. Preprocessing data with normalisation in CNNs enhances convergence, stabilises training and improves generalisation by scaling input features to zero mean and unit variance. The dataset underwent augmentation through the process of padding the image with an additional 4 pixels on all sides, followed by the random sampling of a 32x32 crop from the padded image. Additionally, random horizontal flips are applied to the 32x32 cropped image. Subsequently, the CutOut technique which was proven to improve the regularisation of CNN by DeVries and Taylor [2017] is employed on the flipped image as a final step.

2.6.2 Multi-step Learning Rate

MultiStepLR is a learning rate scheduler commonly used in deep learning frameworks such as PyTorch. It reduces the learning rate at predefined epochs during training by a decay value. This method stabilises training and aids model convergence by dynamically adjusting the learning rate based on the progression of training. EfficientNetV2 [Tan and Le, 2021] achieved state-of-the-art performance while implementing the MultiStepLR learning rate scheduler in the training phase to optimise the architecture search process. In GeneticNAS, the learning rate decreases by a factor of 10 at $N_e = 50$ and 75. N_e is the number of epochs in percentage. $N_e = 50$ for 100 epochs will be epoch 50.

2.6.3 Dropout and drop-path

The Dropout technique [Srivastava et al., 2014] prevents overfitting by randomly dropping out (setting to zero) a proportion of neurons during training. This technique mitigates the overfitting phenomenon and promotes model generalisation by diminishing neuron interdependence, fostering the learning of diverse features and mitigating model reliance on specific neurons.

The drop-path technique as specified in Gastaldi [2017] is another regularisation technique commonly utilised in deep networks to overcome the overfitting problem.

2.7 Potential Optimisation

Extensive research in similar EA-based NAS has revealed a prevalent trend where NAS architectures prioritise maximal accuracy while neglecting concerns regarding power consumption. Such an approach is unsustainable in the long term. Recognising this gap, I discovered the significant potential in utilising the inherent randomness characteristic of genetic algorithms to alleviate computational overhead. Despite being commonly overlooked in the field, this utilisation of randomness presents a promising avenue for addressing sustainability challenges.

Chapter 3

Methodologies

In this chapter, an initial exposition on GeneticNAS adapted from Habi and Rafalovich [2019] is presented. An overview of the implementation of GeneticNAS is presented. Subsequently, the experimental procedures of this project which were used to optimise the genetic algorithms of GeneticNAS are outlined. The experimental objective is to optimise accuracy without significantly compromising temporal and resource efficiency. Finally, a formula for assessing the trade-off between accuracy and efficiency is discussed.

Section 3.1 provides an outline of the implementation of GeneticNAS solely attributed to Habi and Rafalovich [2019]. This provides insights into the overall framework so the experiments proposed in section 3.2 can be comprehended.

3.1 GeneticNAS

GeneticNAS proposed by Habi and Rafalovich [2019] is essentially a combination of ENAS and EA-based NAS. It integrated genetic algorithms search with Stochastic Gradient Descent(SGD) in order to find the accurate NN architecture for a given dataset. The rationale for picking GeneticNAS was detailed in Section 1.

3.1.1 Search Space

The convolutional search space is predicated upon the recurrence of identical cellular structures. It is a prevalent methodology utilised in state-of-the-art neural network designs such as MobileNetv2 [Sandler et al., 2019], Inception-v4 [Szegedy et al., 2016] and residual deep learning [He et al., 2015].

The cell draws inspiration from Pham et al. [2018], which employs a Directed Acyclic Graph (DAG) representation incorporating a specified number of blocks N_b.

Each block is characterised by four integer representations, specifying its operational functionality and inter-block connectivity within the cell. The initial pair of integers, denoted as i_A and i_B, signify the inputs of the block. The subsequent pair, j_A and j_B, represents the operation type associated with each input.

i_A	i_B	j_A	j_B
5	2	1	3

Figure 3.1: i_A and i_B denotes Input A and Input B. j_A and j_B denotes operator of Input A and operator of Input B.

Each block within this DAG representation comprises two inputs and two operators which culminates with a merge operation in the end using addition.

Block 1	Block 2	Block 3	Block 4	Block 5	.....	Block N
3 1 2 3	0 4 1 0	5 2 3 2	1 0 4 3	4 3 0 1	.....	InputA InputB Op1 Op2

Figure 3.2: DAG representation. An integer list in blocks representing a cell type

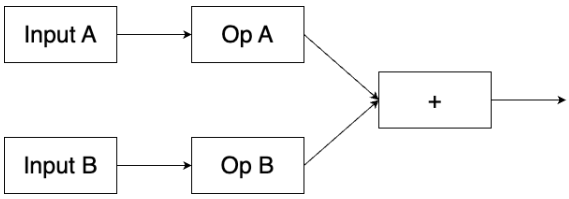

Figure 3.3: Computation within a block. Two inputs undergo computations according to two operators and culminate with a merge operation

The operators utilised in GeneticNAS is the same as Pham et al. [2018]. Specified as follow:

- identity
- 3x3 max pooling
- 3x3 average pooling
- 3x3 depth-wise-separable convolution
- 5x5 depth-wise-separable convolution

The depthwise separable convolution operation is presented in Figure 3.4. DW-Convolution denotes depth-wise convolution, PW-Convolution denotes point-wise convolution and BN denotes batch normalisation[Ioffe and Szegedy, 2015]. To enhance depth, this convolution structure is iterated twice.

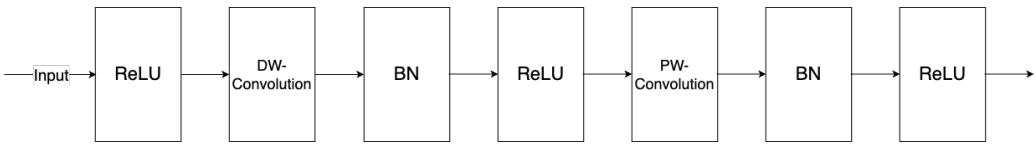

Figure 3.4: Depthwise separable convolution operation framework

The pooling operation is presented in Figure 3.5.

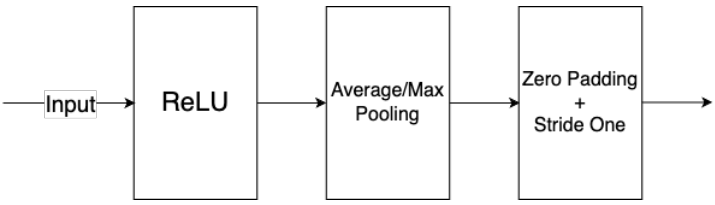

Figure 3.5: Pooling operation framework

Furthermore, each input integer encapsulates every conceivable connection to the block, including inputs from the DAG. Each block exclusively accepts input from the block that directly descends from it. Illustratively, the third block has the capability to receive input from Block 0, Block 1, and Block 2 as well as from the input of the DAG.

The DAG ultimately concatenate all blocks that are not utilised as the input to any other blocks. Subsequently, a 1x1 convolution is employed to adjust the output channel count to match the channel count within the DAG operation. All operations within the DAG maintain a consistent number of channels. It can be formulated as equation 3.1.

$$h_n = OP^{A}_{i_A,j_A}(h_{i_A}) + OP^{B}_{i_B,j_B}(h_{i_B}) \tag{3.1}$$

Weight-sharing is implemented across models, wherein each input possesses a distinct set of weights per operation as depicted by equation 3.1.

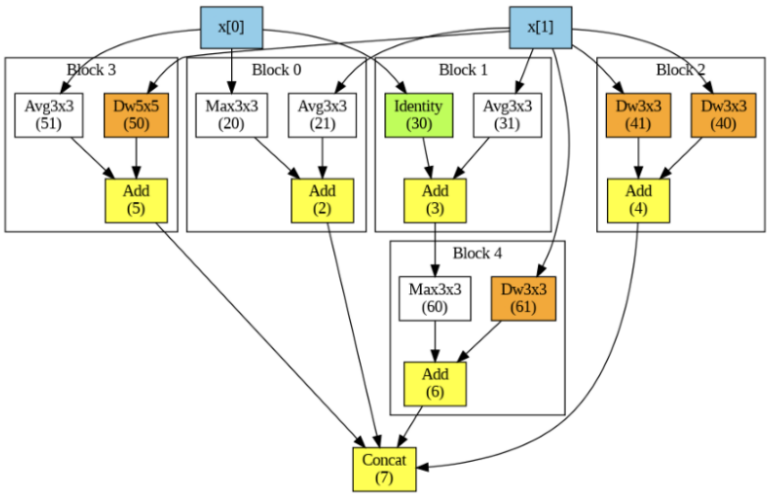

Figure 3.6: 5-Block DAG implementation generated by GeneticNAS

Figure 3.6 is an example 5-Block DAG implementation of GeneticNAAS. Block 3 can be represented by [0 1 2 4].

An inherent drawback of GeneticNAS lies in the exponentially large search space it entails. Specifically, if the convolution cell comprises $N_B$blocks and allows N_{op} operators, the search space encompasses $N_{op}^{N_b} \times (N_b!)^2$ configurations.

3.1.2 Network Architecture

The network architecture is constructed using three types of cells:

- Input cells - 3x3 convolution on its input to expand the number of channels from RGB to N_c.
- Normal cells - Normal cells can have $N_{cells} - 1$ or N_{cells} cells.
- Reduction cells - Average pooling with stride two that is followed by a 1x1 convolution. Effectively expands the number of channels by a factor of two.

The network architecture starts with an input cell, followed by two pairs of normal with reduction cells and concludes with another normal cell.

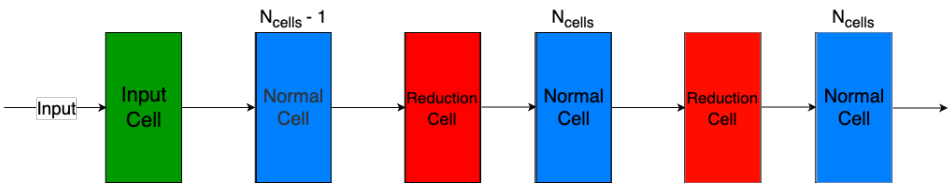

Figure 3.7: Network architecture with three cell types

The normal and reduction cells have h_n and $h_n - 1$ as the input of the DAG, where they denote the output of the previous cell and the input of the previous cell correspondingly. $h_n + 1$ denotes the output of the current cell. All cells have residual connection [He et al., 2015] (denoted as RC in figure 3.8) and 'Squeeze and Excitation' [Hu et al., 2019] block (denoted as SE in figure 3.8).

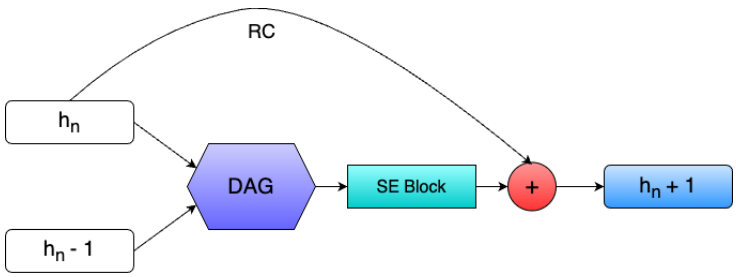

Figure 3.8: Cell structure with Residual Connection and Squeeze and Excitation block

3.1.3 Neural Architecture Search

As previously mentioned, GeneticNAS combines genetic algorithms with SGD to effectively search for an optimal neural network architecture tailored to a specific dataset. The genetic algorithms utilised will first be presented in this subsection and the integration of it into the NAS will be explained subsequently.

Genetic algorithms typically revolve around optimising the solutions within a population. A *population* often refers to a collection of individual candidate solutions or potential solutions to a given optimisation problem. Each candidate of solution within a population is referred to as an *individual*.

3.1.3.1 Selection

In the selection process, the fitness value f_i assigned to each individual within the current population is leveraged to determine the probability of individual selection. This probability is computed by normalising each individual's fitness value against the aggregate sum of fitness values across the population as shown in equation 3.2.

$$p_i = \frac{f_i}{\sum_i f_i} \tag{3.2}$$

N_p individuals are subsequently selected from the current population based on p_i

3.1.3.2 Crossover

The crossover operation involves selecting two individuals referred to as *parents* and generating two new individuals known as *offspring* from them. GeneticNAS proposed by Habi and Rafalovich [2019] incorporated a prevalent technique — uniform crossover along with block crossover. However, only block crossover, which is the superior performing approach between the two was chosen as the baseline model of this work.

Block crossover Block crossover utilised the previously defined network structure where there are N_b blocks. The methodology is initiated by selecting two best-performing individuals as parents. Subsequently, it generates a binary sequence of length N_b by random which indicates whether a crossover operation should occur or not for that block. The two new individuals generated through crossovers are the corresponding offspring of the selected parents.

	Block 1	Block 2	Block 3	Block 4	Block 5
Parent A	3 1 2 3	0 4 1 0	5 2 3 2	1 0 4 3	4 3 0 1
Parent B	2 5 1 2	3 4 0 3	0 2 4 1	5 0 2 4	1 3 3 0
Selector	0	1	1	0	1
Offspring A	3 1 3 2	3 4 0 3	0 2 4 1	1 0 4 3	1 3 3 0
Offspring B	2 5 1 2	0 4 1 0	5 2 3 2	5 0 2 4	4 3 0 1

Figure 3.9: Demonstration of Block crossover

Assume the randomly generated selector is [0, 1, 1, 0, 1], meaning block 2, block 3 and block 5 of the parents are selected for crossover operation to take place. The effect of the crossover operation can be seen on offspring as demonstrated in Figure 3.9.

Blend crossover Blend crossover implemented by me will be discussed further in the following experiment section 3.2. The results in section 4 demonstrate its superior performance compared to the original block crossover method implemented in GeneticNAS, with some minimal compromise on efficiency.

3.1.3.3 Mutation

Mutation induces random modifications to the individual's representation. GeneticNAS employs signed flip mutation. Initially, a binary value is generated for each gene within an individual based on a mutation probability p_m to select genes to be mutated. The higher p_m is the more likely 1 will be selected instead of 0. Subsequently, a signed value is generated uniformly random to be either -1 or 1. The mutation operation is represented in equation 3.3.

$$\text{New value} = \text{current value} + v_{flip} \times v_{sign} \tag{3.3}$$

where v_{flip} denotes flip value andv_{sign} denotes sign value.

The values after the mutation operation are clipped within the range of 0 and max values of that gene. The fourth gene in Block 2 of Figure 3.10 has undergone mutation operations and the new value is supposed to be -1. However, it will instead be assigned the maximum possible value for that particular gene which is 4. Generally, if the computed value is less than 0, it will be assigned the maximum value, and if the computed value exceeds the maximum value, it will be assigned 0.

	Block 1				Block 2				Block 3				Block 4				Block 5			
Parent	3	1	2	3	0	4	1	0	5	2	3	2	1	0	4	3	4	3	0	1
Flip	0	0	1	1	0	0	0	1	0	1	0	0	1	0	1	0	1	1	1	0
Sign	1	-1	1	1	-1	-1	1	-1	-1	-1	1	-1	-1	1	-1	1	-1	-1	-1	1
Offspring	3	1	3	2	0	4	1	4	5	1	3	2	0	0	3	3	3	2	4	1

Figure 3.10: Demonstration of signed flip mutation

3.1.3.4 Replacement

In the scope of EA, the population size N_p is typically maintained throughout all iterations. When new individuals are inserted into the current population pool, the same amount of worst-performing individuals will be removed. GeneticNAs employed the steady-state-no-duplicates method detailed in [Burke and Kendall, 2005]. This methodology ensures the population comprises the most proficient N_p individuals from both the current population and the subsequent generation.

3.1.3.5 Integration of genetic algorithms

The weight-sharing attribute of one-shot NAS is implemented in GeneticNAS, where subsequent child networks are trained using the same weight. Weight sharing necessitates the adaptability of model coefficients to changes in network architecture, where updates to all the weights are mandatory. In order to achieve such weight updates, GeneticNAS samples a different architecture in the population pool according to Algorithm 1. Taken directly from Habi and Rafalovich [2019]:

Algorithm 1: GeneticNAS Sampling Process

Data: Current population x_p
Result: Sampled individual x_s
$x_0, x_1 \leftarrow$ Sampling(x_p);
$x_0, x_1 \leftarrow$ Crossover(x_0, x_1);
$x_s \leftarrow$ Mutation(x_0);

The sampling process effectively samples two individuals from the current population x_p, performs crossover on the sampled $x_0$and x_1, then mutates the first offspring x_0 and inserts the result into the population as shown in Algorithm 1.

After the training epochs are completed, a subsequent generation is generated from the existing population through a sequence of three stages: selection, crossover, and mutation detailed earlier. Thus, each training epoch has two generations of genetic algorithms embedded to update the population pool. Finally, each individual undergoes evaluation on a subset of the validation set with a size of N_v. Subsequently, upon completion of the evaluation process for all individuals, the population will be updated using the aforementioned replacement strategy. Evaluating on a subset N_v rather than the complete set reduces computation overhead and allows more generations of EA to take place.

Algorithm 2: GeneticNAS Algorithm Integration. Taken from Habi and Rafalovich [2019].

```
Data: Dataset
Result: Network architecture
Initialization weights;
for each epoch do
    for each batch do
        Sample a random child network from the population;
        Perform forward pass;
        Perform backward pass;
    end
    Generate the next generation from the population;
    for each child in the next generation do
        Set child architecture;
        Evaluate architecture accuracy;
    end
    Update population using replacement;
end
```

Algorithm 2 taken directly from Habi and Rafalovich [2019].

3.2 Experiments

The experiments are conducted on CIFAR10 and CIFAR100 datasets, they are pre-processed and augmented as detailed in chapter 2. I have chosen the experiments conducted by Habi and Rafalovich [2019] as the baseline experiment models due to the fact that the experimental parameters were derived from an ablation study and demonstrated optimal suitability for the proposed network architecture. However, the training epochs are reduced to 100 due to computational resources and time constraints to train such a deep network. The training and evaluation duration for each CIFAR10 model amounts to approximately 12 hours, whereas it extends to around 16 hours for the CIFAR100 model on a V100 GPU. The baseline experiment parameters are as follows:

Parameter	Value
batch_size	128
batch_size_val	1000
n_epochs	100
n_blocks	2
n_block_type	3
n_nodes	5
n_channels	20
generation_size	20
generation_per_epoch	2
full_dataset	false
population_size	20
keep_size	0
mutation_p	0.02
p_cross_over	1.0
cross_over_type	Block
learning_rate	0.1
lr_min	0.0001
weight_decay	0.0001
dropout	0.2
drop_path_keep_prob	1.0
drop_path_start_epoch	50
cutout	true
n_holes	1
length	16
LRType	MultiStepLR
num_class	10
momentum	0.9
aux_loss	false
aux_scale	0.4

(a) CIFAR10 baseline experiment parameter values

Parameter	Value
batch_size	64
batch_size_val	1000
n_epochs	100
n_blocks	2
n_block_type	3
n_nodes	5
n_channels	48
generation_size	20
generation_per_epoch	2
full_dataset	false
population_size	20
keep_size	0
mutation_p	0.01
gaussian_s	0
p_cross_over	1.0
cross_over_type	Block
learning_rate	0.05
lr_min	0.0001
weight_decay	0.0001
dropout	0.2
drop_path_keep_prob	1.0
drop_path_start_epoch	50
cutout	true
n_holes	1
length	16
LRType	MultiStepLR
num_class	10
momentum	0.9
aux_loss	false
aux_scale	0.4

(b) CIFAR100 baseline experiment parameter values

Figure 3.11: CIFAR10 and CIFAR100 baseline experiment parameter values

The following models are implemented to optimise the EA of the baseline models:

- Gaussian Flip
- Uniform Random
- Adaptive Flip
- Blend crossover

I will discuss implementations and rationales individually in the following subsections.

Multiple versions of proposed models may exist, with each subsequent version exhibiting superior performance in terms of accuracy compared to its predecessors.

3.2.1 Gaussian Flip

When considering optimisation strategies for the genetic algorithm, mutation operations may naturally come to mind as an initial focal point. The rationale of the Gaussian flip methodology is based upon the introduction of additional yet minimal randomness into the original mutation operation. This allows more granular exploitation into the neighbourhood solutions without introducing too much chaos and disrupting stability.

Gaussian Flip v.1 Gaussian Flip v.1 is used in conjunction with the original flip mutation. It first mutates the genes selected by the mutation operation, then performs a weighted average with the mutated value obtained from the original signed flip mutation to derive the final combined mutated value. The gaussian mutation rate $p_{gaussian}$ is selected to be the same as the signed flip mutation rate $p_{mutation}$.

Algorithm 3: Gaussian Flip v.1 Algorithm

Input: Selected block(iv)
Output: Gaussian Mutated Value

mutation_mask ← np.random.rand(iv.shape[0]) $< p_{gaussian}$; mutation_values ← np.random.normal(0, 1, iv.shape[0]);
gaussian_mutated_value ← iv + mutation_values * mutation_mask;
gaussian_mutated_value ← np.clip(gaussian_mutated_value, 0, max_value);
return *Gaussian mutated value*;

It first generates a random array of boolean values mutation mask with probability $p_{gaussian}$ of being true. Subsequently, it generates an array of random numbers ranging (0-1) from a normal distribution as mutation values. Mutation values are then added to genes that are selected by the mutation mask. The gaussian-mutated values are rounded to the nearest integer, resulting in either an addition of 1 or no change and clipped within the range of the max value of that gene or 0. Finally, The selected block of genes(iv) is mutated weighted on both the signed flip and gaussian mutation 3.4:

$$iv_{combined} = (p_{flip} * iv_{flip} + p_{gaussian} * iv_{gaussian})/(p_{flip} + p_{gaussian}) \tag{3.4}$$

Gaussian Flip v.2 Gaussian Flip v.2 performs the exact same procedures as Algorithm 3 but instead of performing both mutations individually and weighted at the end, the selected gene is mutated consecutively by the two mutation methods.

Algorithm 4: Gaussian Flip v.2 Algorithm

Input: Selected Block (*iv*)
Output: Combine Mutated Block ($iv_{combined}$)

iv_flip ← Signed flip mutation(iv);
iv_combined ← Algorithm 3(iv_flip);
return *iv_combined*;

3.2.2 Uniform Random

In contrast to the Gaussian Flip method, Uniform Random mutation aims to augment the exploration of the search space by a greater degree of mutation. This introduces greater stochasticity and injects more diverse individuals into the population. Therefore, it promotes greater diversification in the search for optimal solutions.

Algorithm 5: Uniform Random Algorithm

Input: Selected Block(iv)
Output: Uniform Random Mutated Value

mutation_mask ← np.random.rand(current_value.shape[0]) < $p_{uniform}$;
mutation_values ← np.random.normal(0, 1, current_value.shape[0]);
uniform_mutated_value ← iv + mutation_values * mutation_mask;
uniform_mutated_value ← np.clip(uniform_mutated_value, 0, max_value);
return *Uniform Random Mutated value*;

Similar to Algorithm 4, it first generates a random array of boolean values mutation mask with probability $p_{uniform}$ of being true. Subsequently, it generates an array of random numbers ranging (0 to max_value) from a normal distribution as mutation values. Mutation values are then added to genes that are selected by mutation mask and clipped to be within the range of 0 and max_value. Note, it is used independently instead of in conjunction with signed flip mutation.

3.2.3 Adaptive Flip

This differs from Gaussian flip and Uniform Random mutation which emphasise on exploitation and exploration of the search space correspondingly, Adaptive Flip focuses on the parameter and scheduling of the mutation operation. The ablation study conducted by Habi and Rafalovich [2019] is shown in figure A.1 of appendix A which indicates that the performance of the searched architecture significantly deteriorates when the mutation rate $p_{mutation}$ falls below or exceeds a certain threshold. Adaptive flip is implemented with the goal of assessing whether there lies any favourable range of $p_{mutation}$ to be exploited by dynamically adjusting $p_{mutation}$ during the course of training, instead of using a fixed $p_{mutation}$. Adaptive flip utilises the original signed flip mutation method.

Adaptive Flip v.0 Introduces adaptive mutation rates by adding random values drawn from a normal distribution (np.random.uniform(0, 0.2, iv.shape[0]) to the original mutation probabilities $p_{mutation}$ as shown in equation 3.5.

$$p_{adaptive} = p_{mutation} + \text{adaptive_rate} \times \text{np.random.uniform}(0, 0.1, \text{iv.shape}[0]) \quad (3.5)$$

The adaptive_rate is updated randomly every epoch between the range 0 to 0.8 and can be negative. $p_{adpative}$ will be clipped within the range [0.15, 0.25] which is the optimal range taken from the ablation study figure A.1. Deprecated midway since it induces too much instability and the performance is butchered.

Adaptive Flip v.1 Adaptive Flip v.1 is essentially a scheduling method. Instead of randomly mutating the mutation rate adaptively, the mutation rate is adjusted accordingly based on N_e of MultiStepLR. There were two different implementations tested:

- The epochs where $p_{mutation}$ is adjusted is determined by the MultiStepLR schedule. $p_{mutation}$ is increased by 1.5x of the default value to inject more individuals into the population when the learning rate is reduced by a factor of 10 when the epoch is at N_e = 75 for exploration. It is also decreased by 0.75x of default value at N_e = 90 with the aim of allowing more exploitation at the end.
- $p_{mutation}$ is increased at N_e = 30, 50, 75 and decreased at N_e = 85 with similar reasoning.

Adaptive Flip v.2 The mutation rate $p_{mutation}$ adjustment is now in every generation based on the mean population fitness(fp_mean) relative to the current best population fitness(fp_best). Should the fp_mean fall short of surpassing the target (fp_best) by a specified number of occurrences, denoted by (x), the mutation rate is raised by a certain factor f. This adjustment aims to introduce additional individuals into the population when the population fitness does not improve further, thereby achieving a higher mean population fitness and potentially obtaining better solutions. This was found to be the best-performing mutation operation modification. Therefore, different x and f are tested. The $p_{mutation}$ also resets every 20 epochs and bounded by default $p_{mutation}$ * 1.2 to prevent $p_{mutation}$ increases out of control and significantly deteriorates performance, as shown in figure A.1.

3.2.4 Blend crossover

One might question the possibility of whether utilising other crossover methods will optimise the model after thoroughly exploring mutation techniques.

Blend crossover implementation is inspired by Gwiazda [n.d.] and Herrera et al. [2003], the algorithm taken from Gwiazda [n.d.] is shown in the appendix Algorithm 6.

Algorithm 6: BLX-a-b

```
Input  : Two parents X(t) and Y(t) from a parent pool
Output : Two offspring X(t+1) and Y(t+1)
Assume that X_t is better than Y_t;
for i ← 1 to n do
    d_i ← |x_i(t) − y_i(t)|;
    if x_i(t) ≤ y_i(t) then
        Choose a uniform random real number u from interval < x_i(t) − a·d_i, y_i(t) + b·d_i >;
        x_i(t+1) ← u;
        Choose a uniform random real number u from interval < x_i(t) − a·d_i, y_i(t) + b·d_i >;
        y_i(t+1) ← u;
    end
    else
        Choose a uniform random real number u from interval < y_i(t) − b·d_i, y_i(t) + a·d_i >;
        x_i(t+1) ← u;
        Choose a uniform random real number u from interval < y_i(t) − b·d_i, y_i(t) + a·d_i >;
        y_i(t+1) ← u;
    end
end
```

The blend crossover selects blocks within two individuals as parents and blends their genes to generate crossover-ed offspring. The utilisation of a selector adds randomness in crossover operation much like genetics, while also reducing computation overhead and helping improve efficiency.

Algorithm 7: Individual blend crossover

Input: Individual A *individual_a* and Individual B *individual_b*
Output: Blended individuals - iv_a and iv_b

```
n ← individual_a.get_n_op();
selection ← random integers from 0 to 1 with length n;
iv_a ← [];
iv_b ← [];
max_a ← maximum values vector for individual_a;
max_b ← maximum values vector for individual_b;
for i ← 0 to n − 1 do
    if selection[i] = 0 then
        iv_a.append(individual_a.iv[i]);
        iv_b.append(individual_b.iv[i]);
    end
    else
        iv_a.append(blend_a(individual_a.iv[i], individual_b.iv[i], max_a[i]));
        iv_b.append(blend_b(individual_a.iv[i], individual_b.iv[i], max_b[i]));
    end
end
return iv_a, , iv_b;
```

The detailed algorithm for blend_a and blend_b can be found in appendix algorithm 8 and algorithm 9 correspondingly. The operation can be demonstrated by figure 3.12:

	Block 1	Block 2	Block 3	Block 4	Block 5
Parent A	3 1 2 3	0 4 1 0	5 2 3 2	1 0 4 3	4 3 0 1
Parent B	2 5 1 2	3 4 0 3	0 2 4 1	5 0 2 4	1 3 3 0
Selector	0	1	1	0	1
Offspring A	3 1 2 3	1 4 1 2	4 2 4 2	1 0 4 3	3 3 2 1
Offspring B	2 5 1 2	2 4 1 2	3 2 3 2	5 0 2 4	5 3 1 1

Figure 3.12: Demonstration of blend crossover

For Block 5 in figure 3.12, the selector selected 1 meaning blend crossover should take place. Every gene within block 5 of parent A (X(t)) and parent B (Y(t)) will undergo blend crossover. The blocks X(t)=[4, 3, 0, 1] and Y(t)=[1, 3, 3, 0]. The absolute difference d_i = [3, 0, 3, 1] is calculated by $\|X(t)_i$ - $Y(t)_i\|.For i = 1$:

$0.5 \times d_1 = 0.5 \times 3 = 1.5$ and $0.1 \times d_1 = 0.1 \times 3 = 0.3$. Therefore, u_1 is chosen from the interval $\langle X(t)_1 - 1.5, Y(t)_i + 0.3\rangle = \langle 4 - 1.5, 1 + 0.3\rangle = \langle 2.5, 1.3\rangle$ randomly. Assuming 2.7 is chosen, it will be rounded to 3, hence why block 5 of offspring A begins with 3. This will be iterated for all blocks that are selected by the selector and two offspring will be generated from two parents every time during this process.

Chapter 4

Results

In this chapter, an initial overview of all the proficiently trained models is provided, followed by a thorough evaluation of performance on the CIFAR10 and CIFAR100 datasets separately. The evaluation will be critically analysed based on metrics such as validation accuracy, model complexity, and search efficiency.

The result section is structured as follows:

1. CIFAR10
   1.1. Validation Accuracy
   1.2. Model Complexity
   1.3. Efficiency
2. CIFAR100
   2.1. Validation Accuracy
   2.2. Model Complexity
   2.3. Efficiency

Note: The experiments proposed in section 3.2 are the *training search stage*, once the optimal model architecture is found from the 100 training search stage epochs, the found model will undergo another 100 *final search stage* epochs to evaluate the model's performance where no genetic algorithms or NAS operations are involved.

The validation accuracy of models for both the training search stage and the final search stage will be discussed in this chapter. However, the accuracy that would be evaluated for efficiency is the validation accuracy of the best-performing model in the final search stage. The objective is to assess the performance of the discovered architecture **A*** obtained from the training search phase, through subsequent training on the dataset. This evaluation aims to gauge the model architecture's ability to learn specialised abstract features tailored to the designated dataset.

The evaluation of model complexity is conducted through an analysis of the total count of trainable parameters within the best model obtained during the final search stage as detailed in Chapter 2. This selection is made because model accuracy is assessed based on the optimal performance achieved during this final search stage. Comparing the count of trainable parameters for model architectures identified during the training search stage is deemed futile as these parameters will undergo further training during the final search stage and increase. Conventionally, more complex models tend to yield superior accuracy outcomes. Therefore, the primary objective of an effective NAS system should be the acquisition of higher accuracy models with minimal increments in model complexity.

Efficiency is evaluated by **time** in seconds. The term **time** denotes the elapsed time in seconds required for the training search phase of the NAS system to identify a model achieving validation accuracy surpassing specified **thresholds** – 88% for CIFAR10 and 71% for CIFAR100 respectively. The rationale behind this metric lies in that the goal of optimising the genetic algorithms is to enhance the efficiency of the architecture search phase. Therefore, the success of this optimisation hinges on whether the genetic algorithm modifications indeed reduce the time needed to discover a model architecture capable of achieving superior or comparable accuracy to the baseline. The selection of the specified percentage is based on the observation that all models exceeded this threshold during the training search stage. Additionally, choosing a percentage that is too low might introduce variability due to the randomness inherent in the algorithm, potentially compromising the consistency of comparisons if the experiment were to be repeated. Also, less time required is equivalent to fewer computational resources required as they are trained with identical computational power(GPU).

The variables' values will be substituted according to the dataset of interest when performing the calculation. The comparison between the proposed model and the baseline will be quantified in terms of percentage differences in both model complexity and efficiency. The assessment of whether the improved accuracy justifies the heightened model complexity predominantly depends on resource limitations and the objective of resource conservation.

Note: The CIFAR10 model `cifar10_adaptive_flip_v2_107x_4fpm_per20_90_blend` is a hybrid model that combines the two best-performing models, namely `cifar10_adaptive_flip_v2_107x_4fpm_per20_90` and `cifar10_blend_crossover` which utilises a singular methodology (The same applies for the CIFAR100 model). Such hybrid models will also be taken into consideration as it has been found to further enhance performance.

4.1 CIFAR10

4.1.1 Overview of Models

Model Name	Corresponding Methodologies	Parameter Description
cifar10_baseline	Baseline	Baseline
cifar10_adaptive_flip_v2 _102x_3fpm_per20_bound102	Adaptive Flip v.2	f = 1.02, x = 3 $p_{mutation}$ resets at epoch 20, 40, 60, 80; $p_{mutation}$ bounded within default $\times$ 1.02
cifar10_adaptive_flip_v2 _102x_5fpm_per20	Adaptive Flip v.2	f = 1.02, x = 5 $p_{mutation}$ resets at epoch 20, 40, 60, 80
cifar10_adaptive_flip_v2 _1035x_4fpm_per20_90	Adaptive Flip v.2	f = 1.035, x = 4 $p_{mutation}$ resets at epoch 20, 40, 60, 80, 90
cifar10_adaptive_flip_v2 _107x_4fpm_per20_90	Adaptive Flip v.2	f = 1.07, x = 4 $p_{mutation}$ resets at epoch 20, 40, 60, 80, 90
cifar10_blend_crossover	Blend Crossover	Blend crossover
cifar10_adaptive_flip_v2 _107x_4fpm_per20_90_blend	Adaptive Flip v.2 Blend Crossover	f = 1.07, x = 4 $p_{mutation}$ resets at epoch 20, 40, 60, 80, 90; Blend crossover

Table 4.1: CIFAR10 models with names, corresponding methodologies and parameter descriptions

The presentation is restricted to the top seven models to facilitate comparative analysis. Prior examination of the CIFAR100 dataset indicates that Adaptive Flip v.2 exhibits superior performance among mutation strategies. Therefore, further experimentation with various parameters of Adaptive Flip v.2 was conducted using the CIFAR10 dataset as it takes less time to train, thus the great number of occurrences. Initially, the $p_{mutation}$ parameter is constrained to the default value multiplied by 1.02 based on observations that significant deviations from the default value lead to performance degradation. However, it was subsequently determined that this approach proved to be ineffective as it failed to generate a sufficient number of new individuals. It was discovered that utilising x = 4fpm resulted in optimal performance as the mutation rate updated from x = 3 is deemed too aggressive, while x = 5 leads to insufficient new individuals introduced.

4.1.2 Validation

Model Name	Training Search Stage Validation (%)	Final Training Stage Validation (%)
cifar10_baseline	89.12	93.17
cifar10_adaptive_flip_v2_102x_3fpm_per20_bound102	90.38	93.19
cifar10_adaptive_flip_v2_102x_5fpm_per20	89.19	92.55
cifar10_adaptive_flip_v2_1035x_4fpm_per20_90	89.08	92.89
cifar10_adaptive_flip_v2_107x_4fpm_per20_90	88.98	93.19
cifar10_blend_crossover	90.59	93.63
cifar10_adaptive_flip_v2_107x_4fpm_per20_90_blend	91.37	94.03

Table 4.2: CIFAR10 model's validation accuracy

4.1.2.1 Training Search Validation

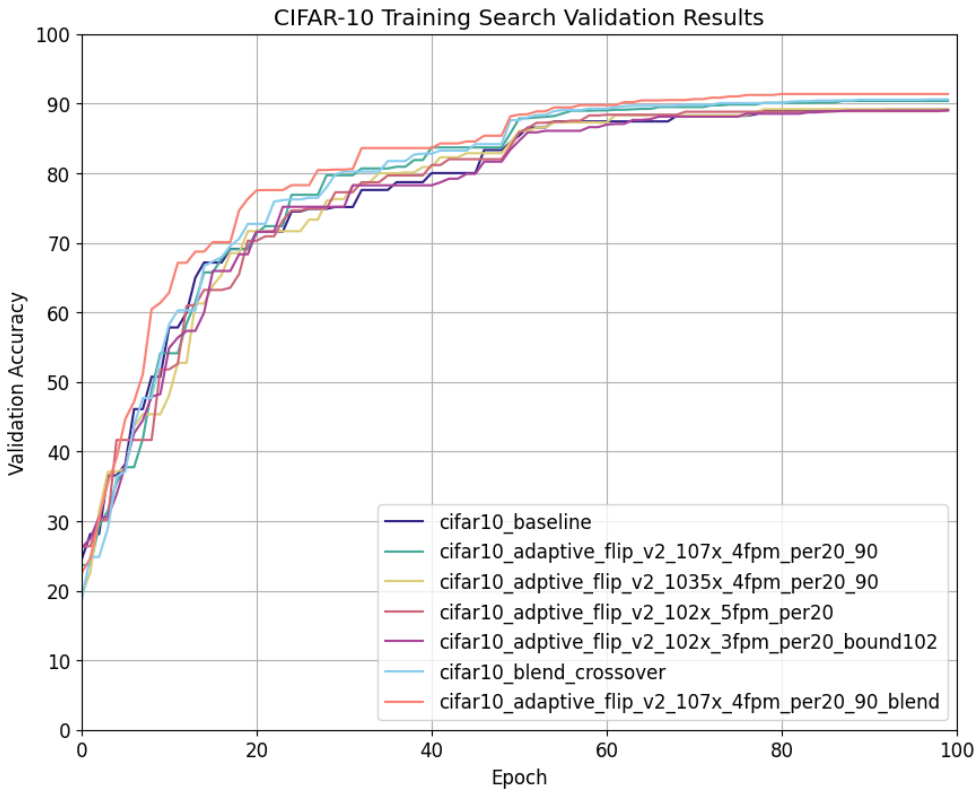

Figure 4.1: Validation Accuracy of different CIFAR10 models in Training Search Stage

Figure 4.1 illustrates that all models with the exception of model `cifar10_adaptive_flip_v2_1035x_4fpm_per20_90`, demonstrate the capability to discover architectural configurations achieving higher validation accuracy than the baseline for CIFAR10 (mostly post-epoch 35). The models `cifar10_adaptive_flip_v2_107x_4fpm_per20_90` and `cifar10_blend_crossover` exceed this threshold, consistently identifying significantly superior architectures compared to the baseline after epoch 25. Notably, the hybrid model `cifar10_adaptive_flip_v2_107x_4fpm_per20_90_blend` consistently outperforms the baseline throughout the entire duration and exhibiting consistently superior architectural discoveries. This highlights the effectiveness of the proposed methodologies in improving the efficiency of the neural architecture search process. These methodologies consistently produce superior-performing models throughout training, with some even outperforming the best model of the baseline at an early stage of the training epoch. One crucial observation is the remarkable efficiency achieved, particularly when computational resources are constrained. Notably, five out of the six proposed models demonstrate the capability to achieve neural network architectures that outperform the baseline, even when the training epoch is halved.

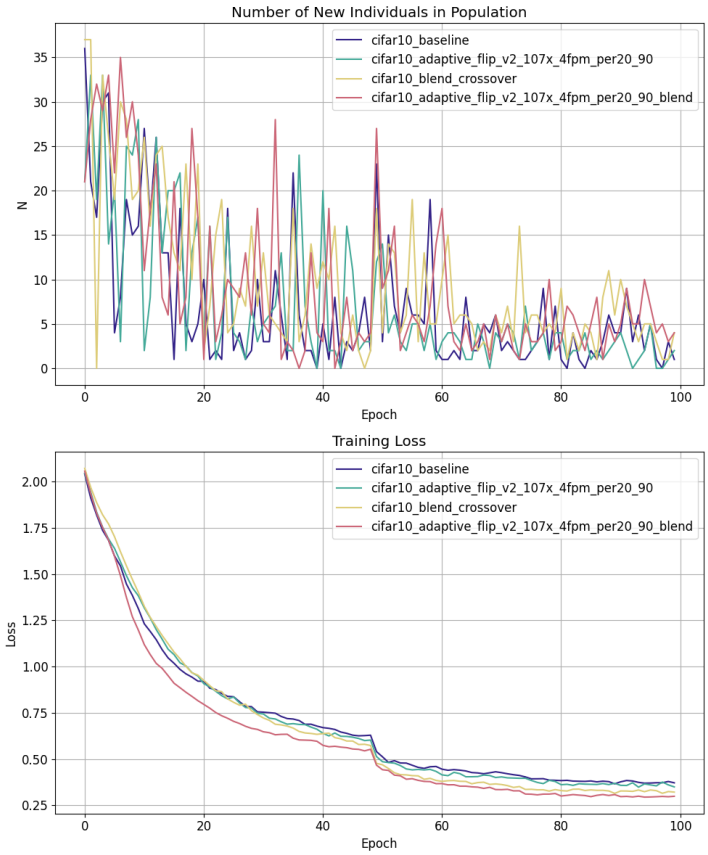

Figure 4.2: Number of individuals added to the population(top) and training loss (bottom) over time during the training search phase. Best 3 performing only for visualisation

Figure 4.2 indicates that the proposed methodologies successfully injected more individuals into the population over time without diminishing the algorithm's capacity to find optimal individuals to classify the CIFAR10 dataset.

4.1.2.2 Final Search Validation

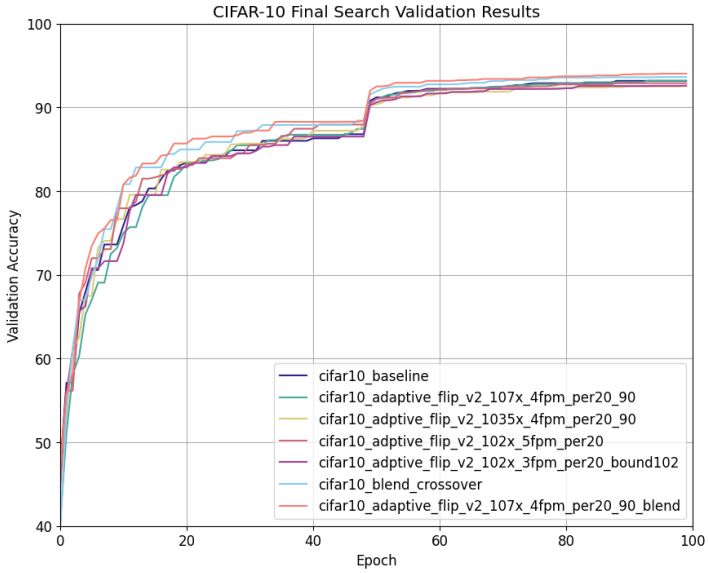

Figure 4.3: Validation accuracy of different CIFAR10 models in final training stage

As shown in figure 4.3, all proposed models ultimately achieved validation accuracy equal to or better than the baseline model. `cifar10_blend_crossover` and `cifar10_adaptive_flip_v2_107x_4fpm _per20_90_blend` performs marginally better than the baseline model throughout the final search stage. All adaptive flip v.2 models except `cifar10_adaptive_flip_v2_102x_3fpm _per20_bound102` attain slightly higher accuracy than the baseline model throughout the final search stage. The huge validation accuracy leap at epoch 50 is attributed to MultistepLR where the learning rate is divided by a factor of 10. I isolated the best 3 performing models for visualisation in Figure 4.4 which clearly demonstrate superior performance over the baseline.

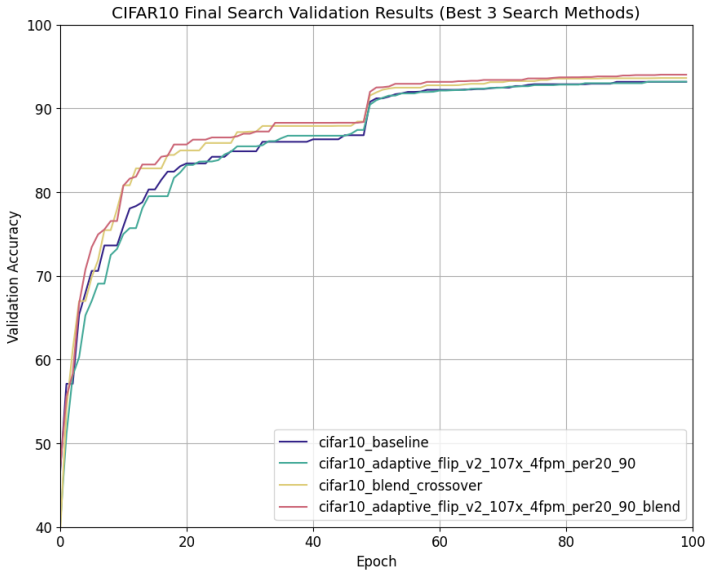

Figure 4.4: Validation accuracy of the best 3 CIFAR10 models in the final training stage

4.1.3 Model Complexity

Model Name	Total Trainable Parameter	% Diff Compared to Baseline
cifar10_baseline	713673	NA
cifar10_adaptive_flip_v2_102x_3fpm_per20_bound102	801753	12.34%
cifar10_adaptive_flip_v2_102x_5fpm_per20	613433	-14.05%
cifar10_adaptive_flip_v2_1035x_4fpm_per20_90	596993	-16.35%
cifar10_adaptive_flip_v2_107x_4fpm_per20_90	649793	-8.95%
cifar10_blend_crossover	965233	35.25%
cifar10_adaptive_flip_v2_107x_4fpm_per20_90_blend	852833	19.50%

Table 4.3: CIFAR10 model's total trainable parameter and % diff compared to baseline

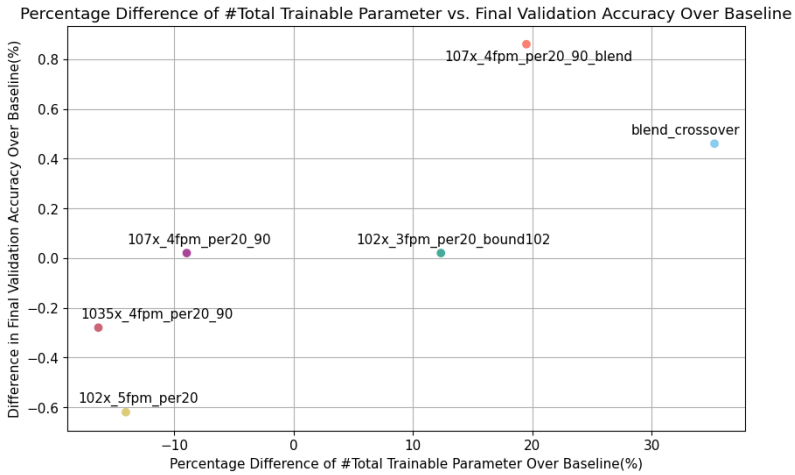

Figure 4.5: % # Total trainable parameters vs % accuracy difference compared to baseline for the proposed CIFAR10 models

It is essential to acknowledge from figure 4.5 that achieving marginally higher validation accuracy with certain models inevitably results in increased model complexity, particularly those employing blend crossover. Whether the trade-off between model complexity and accuracy is worth it is mostly objective-dependent. However, the adaptive v.2 methodology demonstrates the capability to identify models surpassing baseline performance despite possessing a smaller overall count of trainable parameters. This achievement is even more noteworthy, as it showcases the methodology's ability to optimise blend crossover models when used in conjunction to achieve a superior final validation accuracy, all while maintaining a reduced number of trainable parameters.

4.1.4 Efficiency

Model Name	Time(s)	% Improvement Over Baseline
cifar10_baseline	6011	NA
cifar10_adaptive_flip_v2_102x_3fpm_per20_bound102	3634	39.54
cifar10_adaptive_flip_v2_102x_5fpm_per20	3648	39.31
cifar10_adaptive_flip_v2_1035x_4fpm_per20_90	3324	44.70
cifar10_adaptive_flip_v2_107x_4fpm_per20_90	2940	51.09
cifar10_blend_crossover	3089	48.61
cifar10_adaptive_flip_v2_107x_4fpm_per20_90_blend	2706	54.98

Table 4.4: CIFAR10 model's time required to search a model architecture surpassing 88% validation accuracy in training search phase and the % improvement over baseline

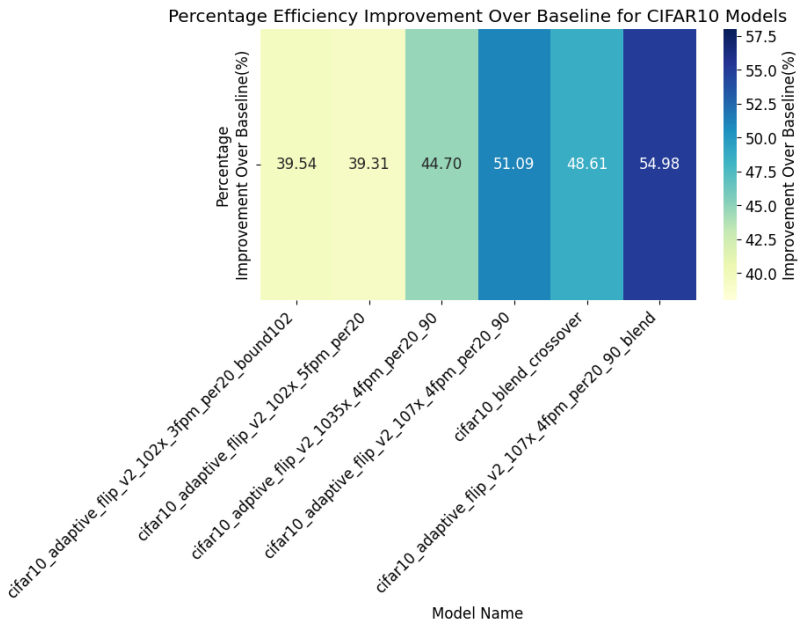

Figure 4.6: % Efficiency improvement of proposed CIFAR10 models over baseline

Figure 4.6 distinctly visualises the proposed model enabling the discovery of a model architecture capable of attaining accuracy above 88% much earlier than the baseline. Notably, the minimal improvement observed exceeds 39% and extends to 55%. The model `cifar10_adaptive_flip_v2_107x_4fpm_per20_90` was identified as the most efficient standalone model. Furthermore, it possesses the capability to optimise the blend crossover method, which further enhances the efficiency when used in conjunction.

4.2 CIFAR100

4.2.1 Overview of Models

Model Name	Corresponding Methodologies	Parameter Description
cifar100_baseline	Baseline	Baseline
cifar100_flip_v2_gaussian001	Gaussian Flip v.2	$p_{gaussian}$ = 0.01
cifar100_flip _uniform_random_0005	Uniform Random	$p_{uniform}$ = 0.005
cifar100_adaptive_flip_v1 _75_150x_90_075x	Adaptive Flip v.1	$p_{mutation}$ $\times 1.5 at N_e$ = 75 $p_{mutation}$ $\times 0.75 at N_e$ = 90
cifar100_adaptive_flip_v2 _107x_4fpm_per20_90	Adaptive Flip v.2	f = 1.07, x = 4, $p_{mutation}$ resets at N_e= 20, 40, 60, 80, 90
cifar100_blend_crossover	Blend Crossover	Blend Crossover
cifar100_adaptive_flip_v2 _107x_4fpm_per20_90_blend	Adaptive Flip v.2 Blend Crossover	f = 1.07, x = 4 $p_{mutation}$ resets at epoch 20, 40, 60, 80, 90; Blend crossover

Table 4.5: CIFAR100 models with names, corresponding methodologies and parameter descriptions

The presentation is restricted to seven models to facilitate comparative analysis; mid-performing models are excluded for streamlining purposes. The inclusion of the Gaussian flip, uniform random, and adaptive flip v.1 models serves to broaden the spectrum of the proposed model despite their inferior performance compared to adaptive flip v.2. This spectrum offers a more comprehensive view of the model landscape.

4.2.2 Validation

Model Name	Training Search Stage Validation (%)	Final Training Stage Validation (%)
cifar100_baseline	71.75	76.21
cifar100_flip_v2_ gaussian001	72.00	77.64
cifar100_flip_v2_ uniform_random_0005	73.82	77.28
cifar100_adaptive_flip_v1_ 75_150x_90_075x	72.81	77.63
cifar100_adaptive_flip_v2_ 107x_4fpm_per20_90	74.26	77.69
cifar100_blend_crossover	74.92	78.69
cifar100_adaptive_flip_v2_ 107x_4fpm_per20_90_blend	74.77	78.69

Table 4.6: CIFAR100 model's validation accuracy

4.2.2.1 Training Search Validation

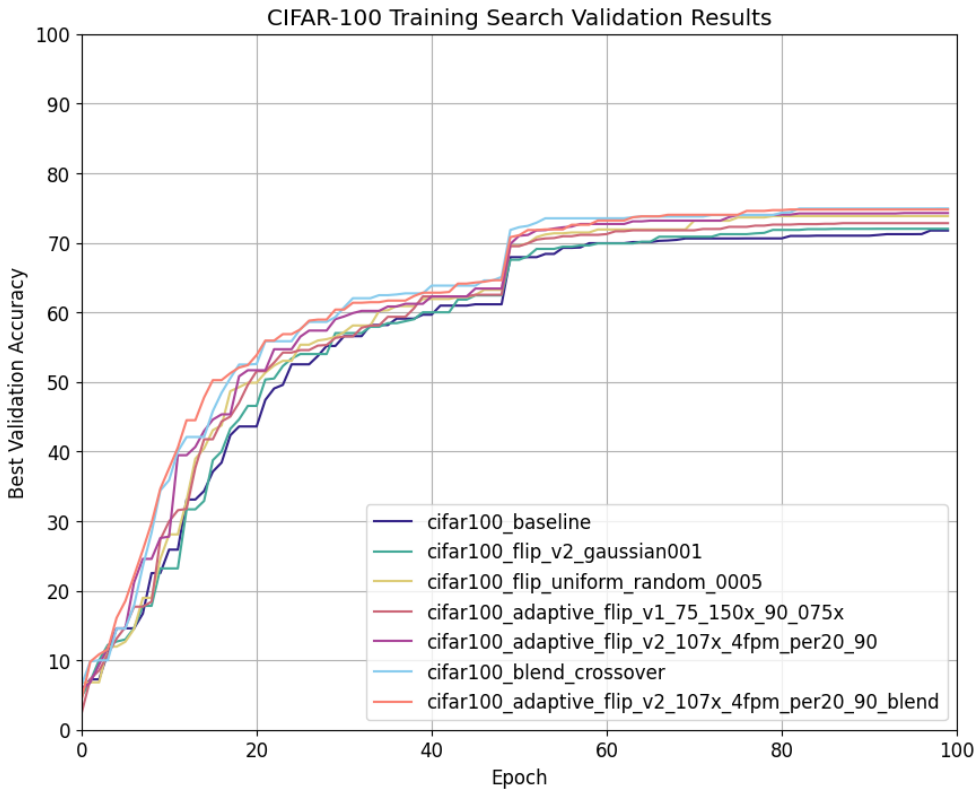

Figure 4.7: Validation accuracy of different CIFAR100 models in training search stage

Figure 4.7 showcases the capacity of all proposed models to identify architectural configurations yielding superior validation accuracy over the baseline for CIFAR100. Specifically, with the exception of the Gaussian flip v.2 model, all models have identified model architectures that exhibit marginal improvement over the baseline across the entirety of the training search phase. The extent of improvement mirrors the pattern observed in CIFAR10, with models employing blend crossover demonstrating a substantial advantage over the others, closely followed by Adaptive Flip v.2 in terms of significance. However, the degree of improvement achieved through the implementation of the proposed methodologies on CIFAR100 is relatively reduced in comparison to what was observed with CIFAR10. This phenomenon may be attributable to the inherent characteristics of CIFAR100 encompasses a significantly larger dataset and models are typically more likely to exhibit lower accuracy. Nonetheless, the proposed model still demonstrates a more effective search.

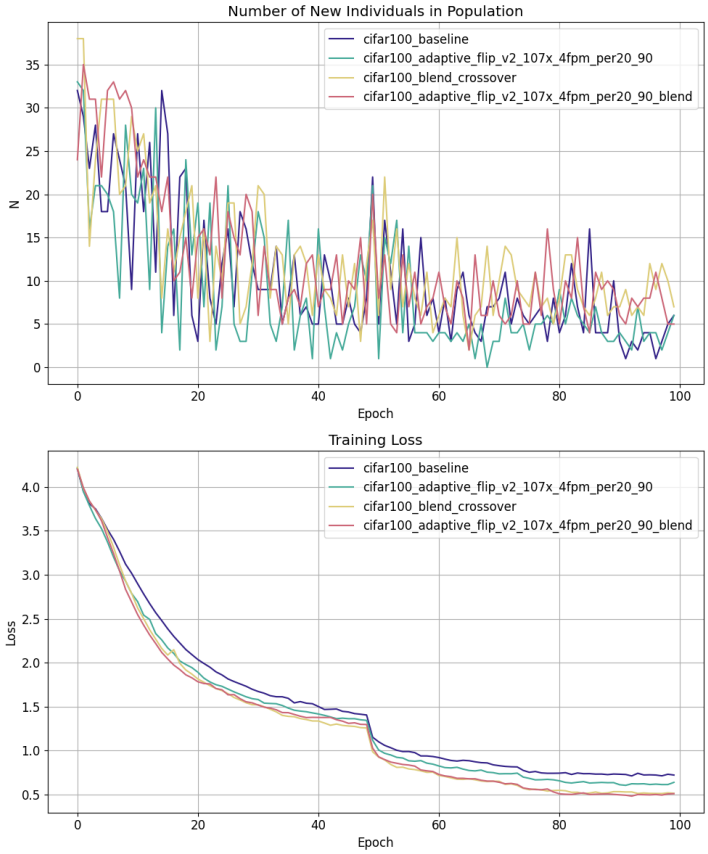

Figure 4.8: Number of individuals added to the population(top) and training loss (bottom) over time during the training search phase. Best 3 performing models only.

Figure 4.8 similar to that of CIFAR10, indicates the proposed methodologies successfully injected more individuals into the population over time without compromising the algorithm's ability to identify optimal individuals for classifying the CIFAR100 dataset. Moreover, it even resulted in a greater reduction in training loss compared to CIFAR10.

4.2.2.2 Final Search Validation

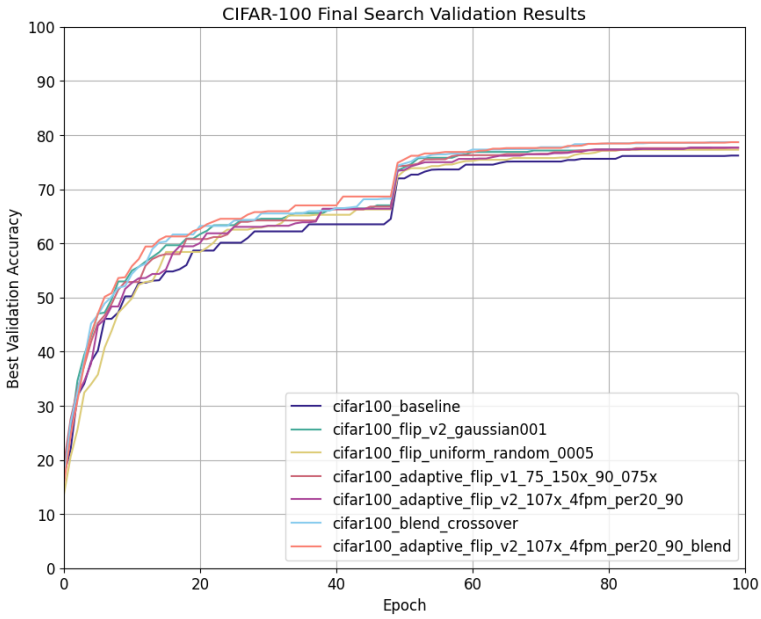

Figure 4.9: Validation accuracy of different CIFAR100 models in the final training stage

Figure 4.9 showcases that all proposed models consistently outperformed the baseline, achieving substantially higher validation accuracy beyond epoch 20 and throughout the remainder of the final training phase. The ultimate validation accuracy rankings of the models are aligned with those established for CIFAR10. Nonetheless, all proposed models can attain validation accuracy that is respectably higher than the baseline. Once again, I isolated the top three performing models for enhanced visualisation where clear improvement is shown in the line plot of figure 4.10.

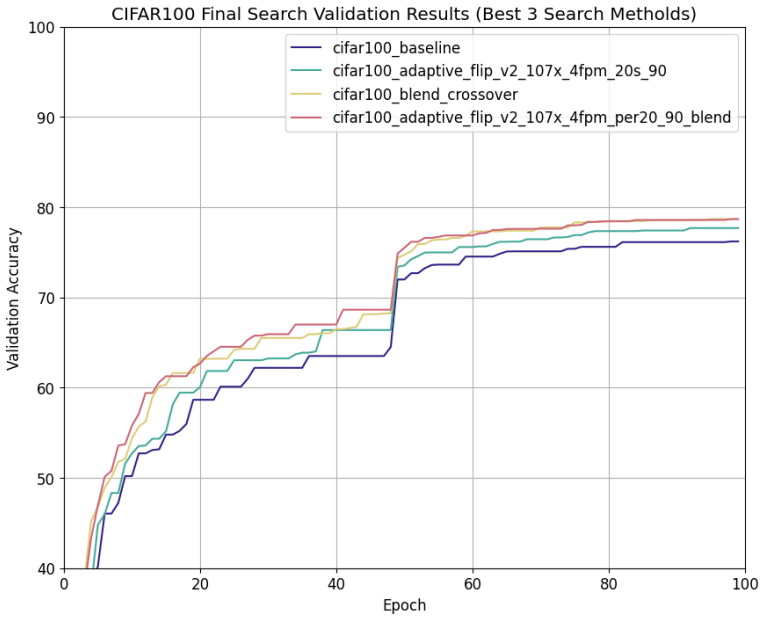

Figure 4.10: Validation accuracy of best 3 CIFAR100 Models in the final training stage

4.2.3 Model Complexity

Model Name	Total Trainable Parameter	% Diff Compared to Baseline
cifar100_baseline	3666760	NA
cifar100_flip_v2_gaussian001	4505984	22.89
cifar100_flip_v2_uniform_random_0005	4225504	15.24
cifar100_adaptive_flip_v1_75_150x_90_075x	3096736	-15.60
cifar100_adaptive_flip_v2_107x_4fpm_per20_90	3787936	3.30
cifar100_blend_crossover	4150336	13.19
cifar100_adaptive_flip_v2_107x_4fpm_per20_90_blend	4505824	22.88

Table 4.7: CIFAR100 model's total trainable parameter and % diff compared to baseline

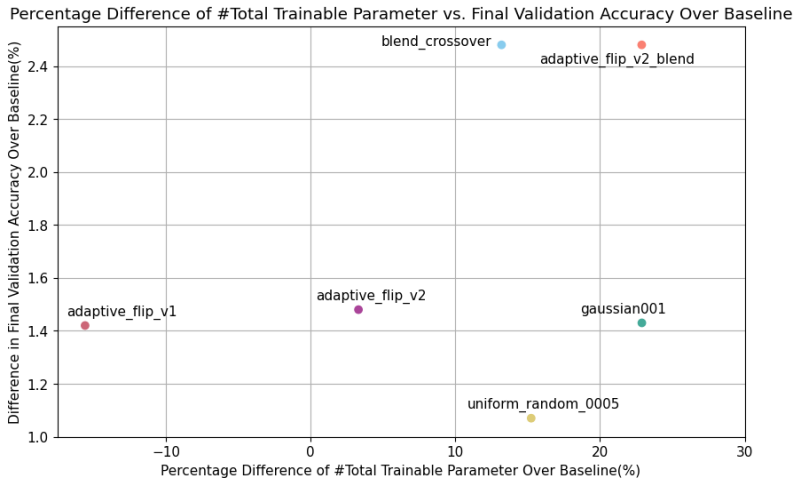

Figure 4.11: % # Total trainable parameters vs % accuracy difference compared to baseline for the proposed CIFAR100 models

Figure 4.11 illustrates the relative percentage variance in model complexity versus accuracy between the proposed model and the baseline. The determination of an acceptable trade-off typically again depends upon the underlying objectives. However, adaptive v.1 achieves a validation accuracy surpassing the baseline model despite having a reduced parameter count. Adaptive flip v.2 on the other hand also achieves greater accuracy with a minimal increment of 3.3% in parameter count. A significant observation is that the utilisation of adaptive flip v.2 in combination with blend crossover did not lead to a reduction in model complexity, contrasting the findings on the CIFAR10 dataset. One conceivable explanation could be that the model architecture is excessively large, and the duration of 100 training epochs may not be sufficient for adaptive flip v.2 to effectively identify models with comparable performance but lower complexity.

4.2.4 Efficiency

Model Name	Time(s)	% Improvement Over Baseline
cifar100_baseline	12175	NA
cifar100_flip_v2_ gaussian001	12123	0.42
cifar100_flip_v2_ uniform_random_0005	8918	26.75
cifar100_adaptive_flip_v1_ 75_150x_90_075x	8456	30.55
cifar100_adaptive_flip_v2_ 107x_4fpm_per20_90	7893	35.17
cifar100_blend_crossover	9118	25.11
cifar100_adaptive_flip_v2_ 107x_4fpm_per20_90_blend	9264	23.91

Table 4.8: CIFAR100 model's required time(s) to search a model architecture surpassing 71% validation accuracy in training Search Phase and its % improvement over baseline

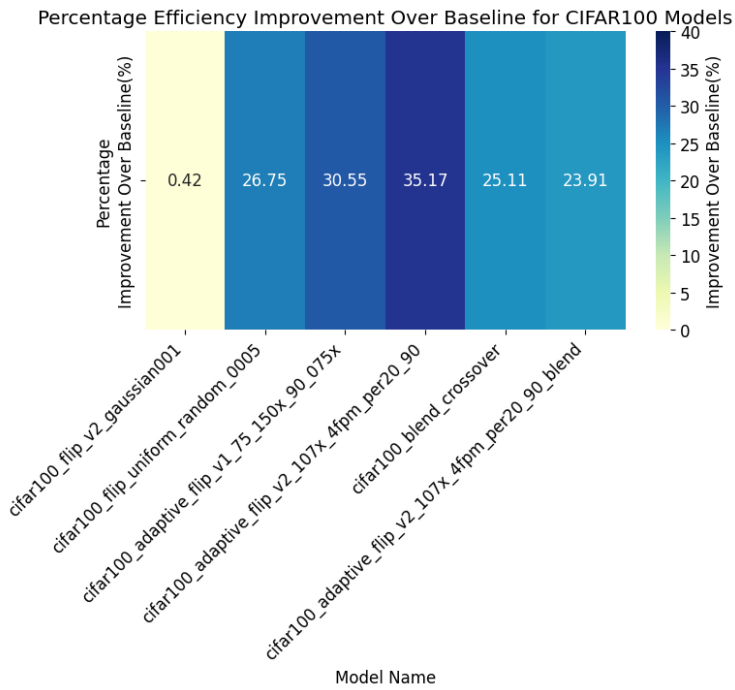

Figure 4.12: % Efficiency improvement of proposed CIFAR100 models over baseline

Figure 4.12 once again clearly visualises how the proposed model enables the discovery of a model architecture capable of attaining accuracy above 71% much earlier than the baseline. The model `cifar100_adaptive_flip_v2_107x_4fpm_per20_90` was identified as the most efficient standalone model, agreeing with the performance in CIFAR10.

Chapter 5

Discussions

5.1 Motivation and Rationale

The primary motivation behind this project stems from the imperative to address the sustainability issues inherent in deep neural networks, it is a pressing concern demanding attention in the foreseeable future. Following an in-depth review and analysis of relevant literature, it became evident to me that optimising the efficiency of the neural architecture search process could offer a partial alleviation of this issue. During this investigation, an evolutionary-based NAS GeneticNAS [Habi and Rafalovich, 2019] was identified. However, the objective of GeneticNAS primarily revolves around maximising accuracy, similar to numerous contemporary state-of-the-art NAS systems. GeneticNAS predominantly emphasises parameter optimization within its proposed evolutionary algorithms, lacking extensive consideration of potentially superior alternative methodologies. However, it serves the purpose of a robust baseline well as it incorporates techniques such as weight-sharing and block-wise crossover to mitigate computational overhead. Nonetheless, I identified the potential of optimising the accuracy and efficiency of the architecture search process through optimisation of the genetic algorithms utilised in an EA-based NAS system. This is significant as such investigations are lacking within the field.

I've proposed three mutation optimisation strategies to optimise GeneticNAS's mutation method. Each strategy targets different areas but shares the common goal of improving search effectiveness within the search space. These strategies aim to inject additional individuals into the population without compromising efficiency. The objective is to strategically add just enough individuals to the search without adversely affecting performance or hindering convergence. Conducting an investigation in three areas instead of focusing on one aims to provide a comprehensive understanding of mutation, a key aspect in evolutionary algorithms. This approach highlights how different areas of evolutionary algorithms can offer varying degrees of efficiency improvement when optimised. Furthermore, I've implemented blend crossover which allows more precise gene-to-gene crossover while maintaining the randomised block-wise approach. This leverages GeneticNAS's distinctive cell-like network architecture structure and maximises accuracy optimisation without significant compromise in efficiency.

5.2 Discoveries

All proposed methodologies effectively improved the final validation accuracy of the trained model using the architecture derived from the architecture search phase to varying extents while concurrently enhancing the efficiency of the search process. Among these, Adaptive Flip v.2 demonstrated the highest efficacy, attaining an architecture that yielded superior final validation accuracy compared to the baseline model while reducing model complexity by 8.95%. It exhibited a notable efficiency enhancement of 51.1% during the training search phase for CIFAR10. Moreover, Adaptive Flip v.2 demonstrated strong generalisation capabilities when applied to the CIFAR100 dataset. It achieved a notable 1.48% enhancement in final accuracy compared to the baseline while experiencing only a marginal increase of 3.3% in model complexity. Additionally, during the training search phase, Adaptive Flip v.2 exhibited a substantial efficiency improvement of 35.17% relative to the baseline. This improvement is likely attributable to the hypothesis suggesting the presence of an optimal minor mutation rate adjustment between epochs, particularly during periods when population fitness stagnates and more diverse individuals are required. However, aggressive mutation rate adjustments in the absence of appropriate regularisation techniques can degrade the performance of the search process, leading to the generation of sub-optimal models due to hindered convergence of solutions. This phenomenon is evidenced in the CIFAR10 experiments, underscoring the necessity to identify the optimal adjustment parameter and regularisation strategies through iterative trial and error processes. In terms of accuracy, blend crossover emerges as the most effective among the proposed methodologies, albeit it is accompanied by an increase in model complexity. The impact becomes more prominent in addressing the larger CIFAR100 dataset, where it achieves a notable 2.48% improvement at the expense of a 13.19% parameter increase. Whether this trade-off is justifiable largely depends on the specific task and objectives at hand. However, it is crucial to acknowledge that in the current state-of-the-art NAS for classifying CIFAR100 tasks, models such as ViT [Tan, 2024] with 3.64×10^6 parameters achieve 78.27% accuracy, while CCT-6/3x1 [Hassani et al., 2022] achieves 77.31% accuracy with 3.17×10^6 parameters. This implies a 14.81% parameter increase for a mere 0.96% accuracy improvement. Hence, the observed 2.48% accuracy enhancement with a 13.19% parameter increase is arguably very significant and justifiable.

These two best-performing models are then amalgamated to provide a more comprehensive overview of the proposed methodologies. There is an intriguing discovery arises from their combination. When classifying the CIFAR10 dataset, Adaptive Flip v.2 complements Blend Crossover, resulting in a model with superior final accuracy, reduced model complexity, and enhanced search efficiency. This finding aligns well with previous observations where Blend Crossover enhances accuracy by enabling more granular searches, while Adaptive Flip v.2 optimises the search process. However, the behaviour shifts when dealing with the more complex CIFAR100 dataset. The combined approach yields an architecture with accuracy identical to applying Blend Crossover alone but with a higher parameter count. Additionally, the search efficiency diminishes. This discrepancy may potentially stem from the mutation rate adjusted too aggressively for an exceptionally deep network model. Lowering the adjustment rate could potentially alleviate this issue.

Chapter 6

Conclusions

6.1 Limitation

Due to resource and time constraints associated with training deep neural networks, there are various limitations to the proposed project.

The experimentation with various parameters of Adaptive Flip v.2 was limited to the CIFAR10 dataset. There is no assurance that the larger CIFAR100 dataset will yield identical performance and it necessitates further experiments to evaluate Adaptive Flip v.2's effectiveness on CIFAR100. Nonetheless, the proposed model with its current parameters has already demonstrated significant improvements in accuracy with minimal compromise in model complexity.

The training epochs are capped at 100 and there is no guarantee that extending the training duration would lead to further improvements in model performance or the efficiency gains will sustain. Nonetheless, the findings still demonstrate that within the constraints of limited resources and time, all proposed methodologies succeeded in obtaining architectures with enhanced superior final validation accuracy while improving the efficiency of the search process.

The proposed methodologies are tailored to optimise GeneticNAS. While they have exhibited noteworthy improvements in terms of both efficiency and accuracy within this framework, there is no guarantee that they will be equally effective in optimising other EA-based NAS approaches. Nevertheless, they do offer a promising direction for optimising the trade-off between efficiency and accuracy in such systems.

The proposed strategies have only been tested for utilisation with CIFAR10 and CIFAR100 datasets. Evaluation of their generalisation capacity beyond these datasets was not conducted. Further exploration in this aspect would be valuable for understanding the broader applicability of these methodologies across different datasets and domains.

Additionally, Blend Crossover relies heavily on the cell-like structure of individuals as implemented in GeneticNAS. This crossover method may necessitate modifications to be compatible with other EA-based NAS systems utilising different architectural representations.

6.2 Future work

In order to assess the generalisation of this approach, it could be applied to other datasets beyond CIFAR10 and CIFAR100 or modified to suit domains other than image classification. Such modifications could involve adapting the methodology to handle different types of data or incorporating domain-specific considerations and therefore extend its applicability to a broader range of tasks and domains.

In the current implementation of blend crossover, the parameters α and β which control exploration and exploitation were chosen to be 0.5 and 0.1 respectively. These parameter selections are conservative and represent the middle point. Conducting an ablation study on these parameters may lead to improved results and provide deeper insights into their impact on the crossover process.

Additionally, fine-tuning the parameters of Adaptive v.2 and conducting comparative analyses with state-of-the-art models could significantly enhance its applicability and performance in various domains and scenarios.

Furthermore, Implementing the proposed methodologies within a framework other than GeneticNAS and evaluating their feasibility in alternative frameworks would provide valuable insights into their adaptability and effectiveness across different NAS architectures.

This study primarily emphasises enhancing mutation and crossover strategies to improve search efficiency and ultimately improve final accuracy. However, further optimisation of the selection and replacement phases may potentially yield superior outcomes and further investigations must be conducted to verify this.

6.3 Conclusion

In conclusion, our study demonstrates the efficacy of fine-tuning genetic algorithms within an evolutionary algorithm-based neural architecture search system to improve the trade-off between accuracy and efficiency. Through various proposed methodologies, we observed enhancements in both final validation accuracy and search process efficiency, with Adaptive Flip v.2 emerging as the most promising approach for efficiency while Blend Crossover significantly boosts accuracy albeit increased model complexity. Combining these methodologies yielded intriguing insights, where a synergistic effect was observed in CIFAR10 classification. However, challenges arose in the CIFAR100 classification, suggesting the need for further optimisation, particularly in mutation rate adjustment for deeper networks. Our findings underscore the potential of fine-tuning genetic algorithms within NAS systems to address sustainability concerns and enhance model performance, offering avenues for future research and development in this domain.

Bibliography

Alex. Learning multiple layers of features from tiny images. *University of Toronto*, 05 2009.

Afshine Amidi and Shervine Amidi. Cs 230 - convolutional neural networks cheatsheet, 2019. URL https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks.

Brian Bailey. Ai power consumption exploding, Aug 2022. URL https://semiengineering.com/ai-power-consumption-exploding/.

Bowen Baker, Otkrist Gupta, Nikhil Naik, and Ramesh Raskar. Designing neural network architectures using reinforcement learning, 2017.

Edmund Burke and Graham Kendall. *Search Methodologies—Introductory Tutorials in Optimization and Decision Support Techniques*. 01 2005. ISBN 0-387-23460-8. doi: 10.1007/0-387-28356-0.

Han Cai, Tianyao Chen, Weinan Zhang, Yong Yu, and Jun Wang. Efficient architecture search by network transformation, 2017.

François Chollet. Xception: Deep learning with depthwise separable convolutions, 2017.

Terrance DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout, 2017.

B. Galvan, Greiner D., Périaux J., M. Sefrioui, and G. Winter. Parallel evolutionary computation for solving complex cfd optimization problems : A review and some nozzle applications. *Parallel Computational Fluid Dynamics 2002*, page 573–604, 2003. doi: https://doi.org/10.1016/b978-044450680-1/50072-3.

Eva García-Martín, Crefeda Faviola Rodrigues, Graham Riley, and Håkan Grahn. Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing*, 134:75–88, 2019. ISSN 0743-7315. doi: 10.1016/j.jpdc.2019.07.007. URL https://www.sciencedirect.com/science/article/pii/S0743731518308773.

Xavier Gastaldi. Shake-shake regularization, 2017.

Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2015.

T. Gwiazda. Blend crossover (blx). http://www.tomaszgwiazda.com/blendX.htm, n.d. Accessed on 29th Mar. 2024.

Hai Victor Habi and Gil Rafalovich. Genetic network architecture search, 2019.

Ali Hassani, Steven Walton, Nikhil Shah, Abulikemu Abuduweili, Jiachen Li, and Humphrey Shi. Escaping the big data paradigm with compact transformers, 2022.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

Kaiming He, Georgia Gkioxari, Piotr Dollar, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

Francisco Herrera, Manuel Lozano, and Ana Sánchez. A taxonomy for the crossover operator for real-coded genetic algorithms: An experimental study. *International Journal of Intelligent Systems*, 18:309 – 338, 02 2003. doi: 10.1002/int.10091.

Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017.

Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu. Squeeze-and-excitation networks, 2019.

IBM. What are convolutional neural networks? — ibm, 2023. URL https://www.ibm.com/topics/convolutional-neural-networks.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.

Anders Krogh. What are artificial neural networks? *Nature Biotechnology*, 26:195–197, 2008. doi: 10.1038/nbt1386. Issue Date February 2008.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. volume 521, pages 436–444. Nature Publishing Group, 2015. doi: 10.1038/nature14539. Received 25 February 2015, Accepted 01 May 2015, Published 27 May 2015, Issue Date 28 May 2015.

Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of convolutional neural networks: Analysis, applications, and prospects. *IEEE Transactions on Neural Networks and Learning Systems*, 33(12):6999–7019, 2022. doi: 10.1109/TNNLS.2021.3084827.

Chenxi Liu, Barret Zoph, Maxim Neumann, Jonathon Shlens, Wei Hua, Li-Jia Li, Li Fei-Fei, Alan Yuille, Jonathan Huang, and Kevin Murphy. Progressive neural architecture search, 2018.

Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search, 2019.

Shiqing Liu, Haoyu Zhang, and Yaochu Jin. A survey on computationally efficient

neural architecture search. *Journal of Automation and Intelligence*, 1(1):100002, Dec 2022. doi: https://doi.org/10.1016/j.jai.2022.100002.

Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. Ssd: Single shot multibox detector. *Computer Vision – ECCV 2016*, 9905:21–37, 2016. doi: https://doi.org/10.1007/978-3-319-46448-0_2. URL https://arxiv.org/abs/1512.02325.

Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015.

Kim Martineau. Shrinking deep learning's carbon footprint, Aug 2020. URL https://news.mit.edu/2020/shrinking-deep-learning-carbon-footprint-0807.

Risto Miikkulainen, Jason Liang, Elliot Meyerson, Aditya Rawal, Dan Fink, Olivier Francon, Bala Raju, Hormoz Shahrzad, Arshak Navruzyan, Nigel Duffy, and Babak Hodjat. Evolving deep neural networks, 2017.

Sakib Mostafa and Fang-Xiang Wu. Chapter 3 - diagnosis of autism spectrum disorder with convolutional autoencoder and structural mri images, Jan 2021. URL https://www.sciencedirect.com/science/article/abs/pii/B978012822822700003X.

PapersWithCode. depthwise separable convolution explained, Accessed 2024. URL https://paperswithcode.com/method/depthwise-separable-convolution. Accessed on March 23, 2024.

Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing, 2018.

Waseem Rawat and Zenghui Wang. Deep convolutional neural networks for image classification: A comprehensive review. *Neural Computation*, 29(9):2352–2449, 2017. doi: 10.1162/neco_a_00990.

Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Suematsu, Jie Tan, Quoc Le, and Alex Kurakin. Large-scale evolution of image classifiers, 2017.

Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions, 2021.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks, 2019.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014. URL https://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions, 2014.

Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alex Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning, 2016.

Jen Hong Tan. Pre-training of lightweight vision transformers on small datasets with minimally scaled images, 2024.

Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR, 09–15 Jun 2019. URL `https://proceedings.mlr.press/v97/tan19a.html`.

Mingxing Tan and Quoc V. Le. Efficientnetv2: Smaller models and faster training, 2021.

Neil C. Thompson, Kristjan H. Greenewald, Keeheon Lee, and Gabriel F. Manso. The computational limits of deep learning. *ArXiv*, abs/2007.05558, 2020. URL `https://api.semanticscholar.org/CorpusID:220496063`.

Harsha Jakkanahalli Vishnukumar, Björn Butting, Christian Müller, and Eric Sax. Machine learning and deep neural network — artificial intelligence core for lab and real-world test and validation for adas and autonomous vehicles: Ai for efficient and quality test and validation. In *2017 Intelligent Systems Conference (IntelliSys)*, pages 714–721, 2017. doi: 10.1109/IntelliSys.2017.8324372.

Meiyin Wu and Li Chen. Image recognition based on deep learning. In *2015 Chinese Automation Congress (CAC)*, pages 542–546, 2015. doi: 10.1109/CAC.2015.7382560.

Lingxi Xie and Alan Yuille. Genetic cnn, 2017.

Xiangning Xie, Xiaotian Song, Zeqiong Lv, Gary G. Yen, Weiping Ding, and Yanan Sun. Efficient evaluation methods for neural architecture search: A survey, 2023.

Kaicheng Yu, Christian Sciuto, Martin Jaggi, Claudiu Musat, and Mathieu Salzmann. Evaluating the search phase of neural architecture search, 2019.

Zhao Zhong, Junjie Yan, Wei Wu, Jing Shao, and Cheng-Lin Liu. Practical block-wise neural network architecture generation, 2018.

Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning, 2017.

Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V. Le. Learning transferable architectures for scalable image recognition, 2018.

Appendix A

Appendix

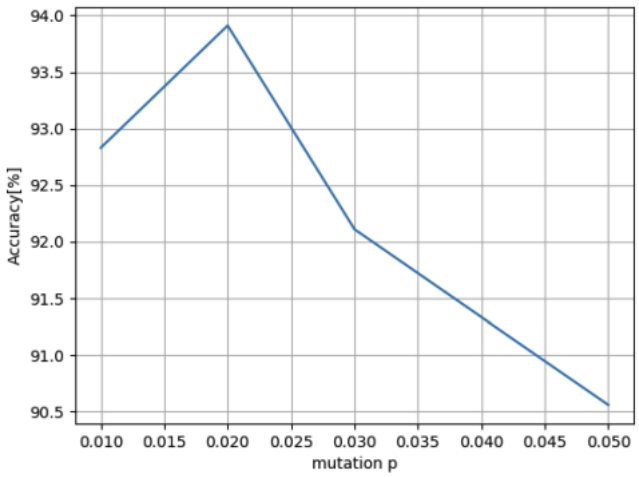

Figure A.1: Ablation study of mutation rate taken from Habi and Rafalovich [2019]

Algorithm 8: Blend individuals a

Input: Two individuals a and b, maximum values max_a
Output: Blended individual $blended_a$

```
blended_a ← [];
for i ← 0 to length(a) − 1 do
    d_i ← |a[i] − b[i]|;
    if a[i] ≤ b[i] then
        lower_bound_a ← a[i] − 0.5 · d_i;
        upper_bound_a ← b[i] + 0.1 · d_i;
    end
    else
        lower_bound_a ← b[i] − 0.1 · d_i;
        upper_bound_a ← b[i] + 0.5 · d_i;
    end
    u_a ← round(uniform(lower_bound_a, upper_bound_a));
    u_a ← max(0, min(u_a, max_a[i]));
    blended_a.append(u_a);
end
return blended_a;
```

Algorithm 9: Blend individuals b

Input: Two individuals a and b, maximum values max_b
Output: Blended individual $blended_b$

```
blended_b ← [];
for i ← 0 to length(a) − 1 do
    d_i ← |a[i] − b[i]|;
    if a[i] ≤ b[i] then
        lower_bound_b ← a[i] − 0.5 · d_i;
        upper_bound_b ← b[i] + 0.1 · d_i;
    end
    else
        lower_bound_b ← b[i] − 0.1 · d_i;
        upper_bound_b ← b[i] + 0.5 · d_i;
    end
    u_b ← round(uniform(lower_bound_b, upper_bound_b));
    u_b ← max(0, min(u_b, max_b[i]));
    blended_b.append(u_b);
end
return blended_b;
```